

JB CSharp

Test cover subtitle

Table of Contents

1. Sources	7
2. Default Note	8
3. Tutorials	9
4. Visual Studio	10
4.1. Solution Explorer	10
5. Syntax	11
5.1. Style	11
5.2. Variables	11
5.3. Conversions	12
5.4. Comments	12
5.5. Data Types	12
5.6. Namespaces	13
5.7. Arrays	14
5.8. Keywords	14
5.9. Escape Characters	14
5.10. Math Operators	14
5.11. Loops	15
5.12. Dialog Boxes	16
5.13. Setting up Class	16
5.13.1. get / set	17
5.13.2. Example	18
5.14. Declarations	19
5.15. Trapping Errors	19
5.16. Properties	20
5.17. If and Switch	20
5.18. Enumeration	21
5.19. Try Catch Finally	21
5.20. Methods	22
5.20.1. Function Parameters	23
5.20.2. Static Methods	24
6. OOP	25
6.1. Example	25
7. Collections	26
7.1. Arrays	26
7.2. List	26
7.3. Arraylist	27
7.4. Stacks	27
7.5. Dictionaries	27
7.6. Queue	28
7.7. Char Arrays	28
7.8. Hashsets	29
8. Strings	32
8.1. String Library	32
8.1.1. Compare	32
8.1.2. Contains	32

8.1.3. CopyTo	32
8.1.4. Format	32
8.1.5. IndexOf	32
8.1.6. IndexOfAny	32
8.1.7. Insert	33
8.1.8. Join	33
8.1.9. LastIndexOf	33
8.1.10. LastIndexOfAny	33
8.1.11. Length	33
8.1.12. PadLeft	34
8.1.13. PadRight	34
8.1.14. Remove	34
8.1.15. Replace	34
8.1.16. Split	34
8.1.17. StartsWith	35
8.1.18. Substring	35
8.1.19. ToCharArray	35
8.1.20. ToLower	35
8.1.21. ToString	35
8.1.22. ToUpper	35
8.1.23. Trim	35
8.1.24. TrimEnd	36
8.1.25. TrimStart	36
8.2. String formatters	36
8.2.1. Currency	36
8.2.2. Decimal	36
8.2.3. Fixed-Point	36
8.3. StringBuilder	36
8.3.1. Append	36
8.3.2. AppendFormat	37
8.3.3. AppendLine	37
8.3.4. Clear	38
8.3.5. CopyTo	38
8.3.6. EnsureCapacity	38
8.3.7. Equals	38
8.3.8. Finalize	38
8.3.9. GetHashCode	38
8.3.10. GetType	39
8.3.11. Insert	39
8.3.12. MemberwiseClone	39
8.3.13. Remove	39
8.3.14. Replace	39
8.3.15. ToString	40

9. Files 41

9.1. Files Library	41
9.1.1. ____	41
9.1.2. AppendAllLines	41
9.1.3. AppendAllText	41
9.1.4. AppendText	41
9.1.5. Close	41
9.1.6. Copy	42
9.1.7. Create	42
9.1.8. Decrypt	42
9.1.9. Delete	42
9.1.10. Dispose	42
9.1.11. Encrypt	42
9.1.12. Open File Dialog	43
9.1.13. Exists	43
9.1.14. File.Replace	43
9.1.15. GetAccessControl	44
9.1.16. GetAttributes	44
9.1.17. GetCreationTimeUtc	44
9.1.18. GetLastAccessTime	44
9.1.19. GetLastWriteTime	44
9.1.20. Open	45
9.1.21. OpenRead	45
9.1.22. OpenText	45
9.1.23. OpenWrite	45

9.1.24. Peek	45
9.1.25. Read	45
9.1.26. ReadAllBytes	45
9.1.27. ReadAllLines	46
9.1.28. ReadAllText	46
9.1.29. ReadLine	46
9.1.30. ReadLines	46
9.1.31. SetAccessControl	46
9.1.32. SetAttributes	47
9.1.33. SetCreationTime	47
9.1.34. SetLastAccessTime	47
9.1.35. SetLastWriteTime	47
9.1.36. Write	47
9.1.37. WriteAllBytes	47
9.1.38. WriteAllLines	48
9.1.39. WriteAllText	48
9.2. StreamReader Writer	48

10. Path 49

10.1. ChangeExtension	49
10.2. Combine	49
10.3. GetDirectoryName	49
10.4. GetExtension	49
10.5. GetFileName	49
10.6. GetFileNameWithoutExtension	49
10.7. GetFullPath	50
10.8. GetInvalidFileNameChars	50
10.9. GetInvalidPathChars	50
10.10. GetPathRoot	50
10.11. GetRandomFileName	50
10.12. GetTempFileName	50
10.13. GetTempPath	50
10.14. HasExtension	51
10.14.1. IsPathRooted	51

11. Directory Class 52

11.1. Methods	52
11.2. CreateDirectory(String)	52
11.3. Delete(String)	52
11.4. EnumerateDirectories(String)	52
11.5. EnumerateFiles(String)	53
11.6. EnumerateFileSystemEntries(String)	53
11.7. Exists	53
11.8. GetAccessControl(String)	53
11.9. GetCreationTime	53
11.10. GetCurrentDirectory	54
11.11. GetDirectories(String)	54
11.12. GetDirectoryRoot	54
11.13. GetFiles(String)	54
11.14. GetFileSystemEntries(String)	55
11.15. GetLastAccessTime	55
11.16. GetLastWriteTime	56
11.17. GetLogicalDrives	56
11.18. GetParent	56
11.19. Move	56
11.20. SetAccessControl	56
11.21. SetCreationTime	56
11.22. SetCurrentDirectory	57
11.23. SetLastAccessTime	57
11.24. SetLastWriteTime	57
11.25. New Note 268	57

12. Controls 58

12.1. Forms	59
-------------------	----

12.1.1. Setup.....	59
12.1.2. Form Close	60
12.1.3. Properties.....	60
12.1.3.1. Cursor	60
12.1.3.2. StartPosition.....	60
12.1.3.3. Text.....	60
12.1.3.4. Enabled	60
12.1.3.5. Size, ClientSize	61
12.1.3.6. Location	61
12.1.3.7. FormBorderStyle.....	61
12.1.3.8. WindowState.....	61
12.1.3.9. BackColor, BackgroundImage	61
12.1.3.10. Opacity	62
12.1.3.11. TopMost.....	62
12.1.3.12. ControlBox,MinimizeBox,MaximizeBox	62
12.1.3.13. Icon	62
12.1.3.14. ShowIcon	62
12.1.3.15. StartPosition.....	63
12.1.3.16. Enabled.....	63
12.1.3.17. ShowInTaskbar.....	63
12.1.3.18. TransparencyKey	63
12.1.3.19. WindowState.....	63
12.1.4. Multiple Forms	63
12.1.4.1. Add new form	63
12.1.4.2. Address Form2.....	64
12.1.4.3. Passing Info	64
12.2. Button.....	64
12.3. Checkbox	65
12.4. CheckedListBox	66
12.5. ColorDialog	66
12.6. ComboBox	66
12.7. ContextMenuStrip	69
12.8. DataGridView	69
12.9. DateTimePicker.....	69
12.10. DomainUpDown.....	69
12.11. ErrorProvider.....	69
12.12. FileSystemWatcher.....	69
12.13. FlowLayoutPanel	69
12.14. FolderBrowserDialog	69
12.15. FontDialog.....	69
12.16. GroupBox.....	69
12.17. HelpProvider	69
12.18. HScrollBar	69
12.19. ImageList	69
12.20. Label	69
12.21. LinkLabel.....	69
12.22. ListBox	69
12.23. ListView.....	71
12.23.1. Links.....	73
12.24. MaskedTextBox.....	73
12.25. MenuStrip.....	73
12.26. MessageBox	73
12.26.1. Y/N Program	75
12.26.2. Dialog Box.....	75
12.27. MonthCalendar	77
12.28. NotifyIcon	77
12.29. NumericUpDown.....	77
12.30. OpenFileDialog	77
12.31. PageSetupDialog	77
12.32. Panel.....	77
12.33. PictureBox.....	77
12.34. PrintDialog	78
12.35. PrintDocument	78
12.36. PrintPreviewControl	78
12.37. PrintPreviewDialog.....	78
12.38. ProgressBar	78
12.39. RadioButton	78
12.40. RichTextBox	78
12.41. SaveFileDialog.....	79
12.42. SplitContainer	79
12.43. Splitter.....	79

12.44. StatusStrip	79
12.45. TabControl.....	79
12.46. TableLayoutPanel.....	79
12.47. TextBox	79
12.48. Timer	80
12.49. ToolStrip	80
12.50. ToolStripContainer.....	80
12.51. ToolTip.....	80
12.52. TrackBar.....	80
12.53. TreeView	80
12.54. VScrollBar	83
12.55. WebBrowser.....	83
12.56. Menus and Toolbars.....	83

13. Common 84

13.1. Random	84
13.1.1. Sample.....	84
13.2. Calling Another Program	85
13.3. Log File	85
13.4. CHM Files	85
13.5. Startup	85
13.6. INI File.....	86
13.6.1. Get Value	86
13.6.2. Change Values.....	86
13.7. Misc.....	87

14. Graphics 88

14.1. Paint Forms.....	88
14.2. Image Manipulation.....	89

1. Sources

General Guides

<http://csharp.net-informations.com/>
<https://www.dotnetperls.com/>

Controls

<file:///M:/bk/Program/cSharp/Controls/srindex.htm>
<http://csharp.net-informations.com/>

Registry Operations

<http://www.codeproject.com/Articles/4808/All-you-wanted-to-know-about-the-Registry-with-C-P>

<msdn.microsoft.com/en-us/library/dd831853> visual studio 2010

<https://visualstudio.microsoft.com/>

[http://msdn.microsoft.com/en-us/library/aa288436\(v=vs.71\).aspx](http://msdn.microsoft.com/en-us/library/aa288436(v=vs.71).aspx)

<http://csharp.net-informations.com/>

<http://www.csharp-station.com/>

<http://www.dotnetperls.com/sort>

<http://csharp.net-informations.com/>

<http://zetcode.com/language/csharptutorial/>

<http://www.java2s.com/Tutorial/CSharp/CatalogCSharp.htm>

//console

//outline only

[C# Tutorial for Beginners \(mmtuts\)](#)

[Visual Studio C# GUI Part 1 \(Ali Farhat\)](#)

[Tutorials Teacher c#](#)

[C# Totorial Tutlane](#)

2. Default Note

3. Tutorials

<https://dotnet.microsoft.com/en-us/learn/csharp>

<https://learn.microsoft.com/en-us/training/modules/csharp-write-first/1-introduction>

<https://www.freecodecamp.org/learn/foundational-c-sharp-with-microsoft/>

<https://www.youtube.com/playlist?list=PLdo4fOcmZ0oULFjxrOagaERVAMbmG20Xe>

<https://www.youtube.com/watch?v=ALXpVIPYqYE>

Exercises

<https://www.w3resource.com/csharp-exercises/conditional-statement/index.php>

Microsoft IO

<http://msdn.microsoft.com/en-us/dotnet/api/system.io.stream>

4. Visual Studio

<https://www.visualstudio.com/vs/community/?NET.desktop.development>

```
cw +tab tab =  
console.WriteLine();
```

Camel casing = myName

Pasquel casing = MyName

A comon style is to start method names with caps and variable names with small letters or camel casing.

Tab Tab commands

ctor = sets up a public method

```
cw = Console.WriteLine();
```

To look up all methods in a class:

Menu View -> Object Browser

Practice screen

menu view > Other windows > C# Interactive
called ripple?

4.1. Solution Explorer

5. Syntax

5.1. Style

Camel caseing - dogCount
Pascal caseing - DogCount

1. toggle bookmarks
2. Saving settings (configuration)

Once windows are set up the way you like them, you can go to tools and export/import settings. You can also reset to default settings.

Control Prefixes

Check box		Chk
Combo box		Cbo
Button		Btn
Form		Frm
Group Box		Grp
Horizontal scroll bar		Hsb
Label		Lbl
List box		Lst
Menu		Mnu
Picture box		Pic
Radio button		Rdo
Text box		Txt
Vertical scroll bar		Vsb

5.2. Variables

A variable is a named placeholder in memory that holds some information.

A variable is declared like so:
<data type> <name>;

All variable names have to start with letters or underscore and can contain only letters, numbers, and '_'.

camelCase Notation

```
string name = "John";  
int    numb = 4;  
bool   redhair = false;  
object myobj = 39;
```

```
object myObj = true;
```

```
MessageBox.Show(myObj.ToString());
```

```
int num = Decimal.ToInt32(y);
```

doubles should be initialized to 0 at start.

5.3. Conversions

From string to int

```
Int.Parse("45");  
Convert.ToString(total);
```

From string to int

```
total.ToString();  
Convert.ToString(total);
```

From string to double

```
double.Parse("45.50");
```

```
DateTime today = DateTime.Now;  
string formattedDate = today.ToString("yyyy-MM-dd");
```

```
ToString("c");    Display currency  
ToString("n2");   Display number of digits after decimal point
```

5.4. Comments

// symbol means ignore rest of line
Can be place before code line or after

Multi line comments syntax

```
/*  
text  
  
*/
```

Xml summary code

```
/// text  
/// text  
/// text  
Describes function which can be read when  
Hovering over function calls
```

5.5. Data Types

Types [Previous](#) [Top](#) [Next](#)

Data Types

Boolean or **bool**. Can only contain two values true and false.

int is short for integer, a data type for storing numbers without decimals. Integers have several data types within C#, depending on the size of the number they are supposed to store.

string is used for storing text, that is, a number of chars. In C#, strings are immutable, which means that strings are never changed after they have been created. When using methods which changes a string, the actual string is not changed - a new string is returned instead.

char is used for storing a single character.

float and **decimal** are used to store numbers which may or may not contain decimals.

Integral Types

Type	Size (in bits)	Range
sbyte	8	-128 to 127
byte	8	0 to 255
short	16	-32768 to 32767
ushort	16	0 to 65535
int	32	-2147483648 to 2147483647
uint	32	0 to 4294967295
long	64	-9223372036854775808 to 9223372036854775807
ulong	64	0 to 18446744073709551615
char	16	0 to 65535

Floating Point and Decimal Types

Type	Size (in bits)	precision	Range
float	32	7 digits	1.5×10^{-45} to 3.4×10^{38}
double	64	15-16 digits	5.0×10^{-324} to 1.7×10^{308}
decimal	128	28-29 decimal places	1.0×10^{-28} to 7.9×10^{28}

String Type

C# Character Escape Sequences

5.6. Namespaces

A namespace is where classes are grouped together. System is a namespace. As too are Collections, Generic, and Text.)

Namespaces are used to hold a number of similar functions and to prevent clashes when two functions with the same name are present.

Namespaces organize classes, interfaces, and structs.

You can have a namespace within a namespace.

Every class that references a method within a namespace must have a using statement.

Source: [Namespaces are used to hold a number of similar functions...](#) (18 December 2013 1:35 AM)

5.7. Arrays

An array is an container that holds a fixed number of data all of one type and the number must be asserted in advance.

It is declared as:

```
string[] names = new string[4] = {s,s,s,s};  
or name[0]="s";
```

5.8. Keywords

5.9. Escape Characters

Escape Sequence	Meaning
\'	Single Quote
\"	Double Quote
\\	Backslash
\0	Null, not the same as the C# <i>null</i> value
\a	Bell
\b	Backspace
\f	form Feed
\n	Newline
\r	Carriage Return
\t	Horizontal Tab

5.10. Math Operators

+ Add
- Subtract
***** Multiply
/ Divide

x += 2 Add 2 to current value of x
x -= 2 Subtract 2 from current value of x
x++ Increment x by 1

Operators in Order of Precedence

Left associativity means that operations are evaluated from left to right. Right associativity mean all operations occur from right to left, such as assignment operators where everything to the right is evaluated before the result is placed into the variable on the left.

Source: [Operators in Order of Precedence](#) (18 December 2013 1:26 AM)

5.11. Loops

if, else if, else statements

== if condition is true

!= if condition is false

```
if(name == "Bob")
{
}
else if (name == "Bill"
{
}
else
{
}
```

if conditional code is one line,
braces are not required

Switch loop

very similar to if procedures

```
switch(textBox1.Text)
{
    case "John":
        MessageBox.Show("Entered John");
        break;
    case "Bill":
        MessageBox.Show("Entered Bill");
        break;
    default:
        MessageBox.Show("Other than John or Bill");
        break;
}
```

For loop

```
for (int i = 0; i<5; i++)
    MessageBox.show(1.ToString());
```

Foreach loop

A foreach loop is used to iterate through the items in a list. It operates on arrays or collections such as ArrayList, which can be found in the System.Collections namespace

```
foreach (<type> <iteration variable> in <list>)
{
    statements;
}
string[] names = {"Adam", "Bob","Joe"}
foreach (string s in names)
    { MessageBox.Show(s); }
```

While Loop

```
int i=0;
while (i<5)
{
    MessageBox.Show(i.ToString());
    i++;
}
```

Do While loop

```
int i = 0;
do {
    textbox1.Text += 1.ToString();
    i++;
}
```

while (i<5);

Unlike other loops it checks after the loop which means it will always execute at least once.

Source: [For loop](#) (18 December 2013 1:30 AM)

5.12. Dialog Boxes

5.13. Setting up Class

A class file can be set up in two ways.

Project->Add Class

Right Click Solution Explorer->Add Class

Make sure resulting window says class

Give it relevant name

To create an instance of the class, or an object, use the format as follows inside the control file:

```
classname instancename = new classname();
```

e.g. MyClass mc = new MyClass();

// Calling program

```
Class1 mc = new Class1();
private void button1_Click(object sender, EventArgs e)
{
    mc.function1();
}
```

namespace Caller

```
{
    class Class1
    {
        public void function1()
        {
            MessageBox.Show("Hello from Class File");
        }
    }
}
```

=====

Example 2

=====

A class is designed to model a real world object.
While it is not required, in general you define one class per file.

To be reusable, a class must be declared as a public class. You can only have access to a class if its declared to be public.

Class file

All classes start with the class keyword followed by the classname. Convention is the first letter of the first and subsequent words are capitalized.

While main is declared as static, most other methods are not. Most methods are defined as public followed by a return type or void. Methods follow the same capitalization conventions as classes.

Caller file

To create the object, the new operator must be used. You first declare a variable of the new class type. That creates the memory for the new object which is referenced by the object name.

```
classname object = new classname();  
Class1 mc = new Class1();
```

The newly created object can be told to do something by the object name, period, method name.

5.13.1. get / set

```
class INI  
{  
    public int CurrentValue  
    {  
        get { return Value; }  
        set { Value = value; }  
    }  
}
```

```
-----  
INI I = new INI;  
ini x = I.CurrentValue;  
I.CurrentValue = 5;
```

=====

Automatic properties

```
class INI {  
    public int CurrentValue { get; set; }  
}
```

=====

Former way

```
in Form1.cs  
textBox2.Text = I.getFolder();  
I.putFolder(textBox2.Text);
```

```
in INI file  
    public string getFolder()  
    {  
        return Folder;  
    }
```

```

public void putFolder(string s)
{
    Folder = s;
}

```

5.13.2. Example

Form1.cs

```

using System;
using System.Collections.Generic;
using System.ComponentModel;
using System.Data;
using System.Drawing;
using System.Linq;
using System.Text;
using System.Threading.Tasks;
using System.Windows.Forms;

namespace test1
{
    public partial class Form1 : Form
    {
        public Form1()
        {
            InitializeComponent();

            private void Button1_Click(object sender, EventArgs e)
            {
                Class1 C = new Class1();
                C.Name = "John";
                C.Age = 28;
                C.HasPet = true;

                C.Greeting();
            }
        }
    }
}

```

=====

Class1.cs

```

using System.Collections.Generic;
using System.Linq;
using System.Text;
using System.Threading.Tasks;
using System.Windows.Forms;

namespace test1
{
    public class Class1
    {
        public string Name;
        public int Age;
        public bool HasPet;

        public void Greeting()
        {
            MessageBox.Show("Hi My name is " + Name + " and my age is " + Age);
        }
    }
}

```

```
}  
}
```

5.14. Declarations

Class

```
Config CF = new Config();  
MyClass mc = new MyClass();
```

List

```
List<string> array = new List<string>();
```

StreamReader StreamWriter

```
System.IO.StreamReader rp;  
rp = new System.IO.StreamReader(INI);
```

StringBuilder

```
StringBuilder sb = new StringBuilder();
```

Open File Dialog

```
OpenFileDialog ofd = new OpenFileDialog
```

<https://www.python.org/dev/peps>

Source: [Class](#) (17 December 2013 9:53 AM)

5.15. Trapping Errors

Types of Errors

1. Design-time Errors
2. Run-time Errors
3. Logical Errors

Errors at Design Time

blue lines - Eg something doesn't exist
red lines - syntax error (e.g. curly brackets and semi)
green lines - Variables is declared but not used

Breakpoints

To add a breakpoint, you just click to the left of a line of code and a red ball will appear. From there F10 will step thru code.

The 'Locals Window' will keep track of all local variables.

Trapping errors via try catch

```
try  
{  
    rtb.LoadFile("C:/test.txt");  
}  
catch (System.Exception excep)  
{  
    MessageBox.Show("File not found");  
}
```

Multiple catch statements can be used to catch different kinds of errors.

Source: [Types of Errors](#) (18 December 2013 1:37 AM)

5.16. Properties

Size of form.

Can be changed simply by dragging

Adding controls

A 'control' is anything user can interact with e.g. button.

To add control, drag from toolbox to location.

Or click on toolbox, click on form, and drag to size if desired.

After that you can drag a control. Lines will appear so that things can be lined up. If you hold down Alt, lines won't appear and you can nudge it anywhere.

Changing properties of the control, e.g. button, go to properties box and change text, color, or whatever.

To change a property of a control with code:

1. Type the control name and period.
2. That results in a dropdown list of all properties.
3. If you then hover over the property name, it will give the type it expects.

E.g.

```
button2.Height = 60;
button1.Text="john";
button1.Enabled = false;
```

Source: [Size of form.](#) (18 December 2013 1:37 AM)

5.17. If and Switch

Definition

An 'if statement' will check if something is true. If it is true, it will do something.

Syntax

```
x = 5;
if (if x == 1)
{
    MessageBox.Show("x is equal to 1" );
}
else if (x == 2)
{
    MessageBox.Show("x is equal to 2" );
}
else { MessageBox .Show("x is neither 1 or 2 ");
}
```

Logical Operators

&& and if
|| or if
== equal
!= not equal

Switch Statements

Sample syntax:

```
switch (x)
{
    case 1:
```

```

code;
break;
case 2:
code;
break;
default :
code;
}

```

Source: [Definition](#) (18 December 2013 1:39 AM)

5.18. Enumeration

An enum is an integer array that uses names to represent each of the integers within the array.
e.g.

```

enum Months
{
    Jan, Feb, Mar, Apr, May, June, July, Aug, Sept, Oct, Nov, Dec
}

```

The values can be on same line, different lines, or staggered.

Normally, the first value is 0. However, if the above you could set Jan = 1, then Feb would become 2 etc. The next element is always the next number by default.

Source: [An enum is an integer array that uses names to...](#) (18 December 2013 1:40 AM)

5.19. Try Catch Finally

The purpose of a 'Catch' statement is to catch an error so that a program does not crash.

Example:

```

private void button1_Click(object sender, EventArgs e)
{
    try
    {
        string[] names = new string[2];
        string s = names[4];
    }
    catch (Exception ex)
    {
        MessageBox.Show(ex.Message);
    }
    finally
    {
        MessageBox.Show("Your code is done");
    }
}

```

In this example, name[4] does not exist. Since there is an error in the try statement, the program looks for a catch statement. If there is no error, the catch code would not be executed.

The catch statement can have a generic message (e.g. program error) or a specific program message explaining the error. In the case above, the message box will display message saying:

"Index was outside the bounds of the array."

The finally statement is optional. If one exists, the code block will be executed whether there was an error or not.

Source: [The purpose of a 'Catch' statement is to catch an...](#) (18 December 2013 1:41 AM)

5.20. Methods

A method is any block of code you plan to execute more than once. They are declared as shown here.

<visibility> <return type> <name>(<parameters>)

1. visibility is optional but if none is declared, it defaults to private.
2. return type can be anything or void.

Visibility

The visibility of a class, a method, or variable tells how this item can be accessed. Most commonly private and public.

public

Can be reached from anywhere.

protected

Members can only be reached from within same class or from a class which inherits from this class

private

Can only be reached by members of same class.

Example:

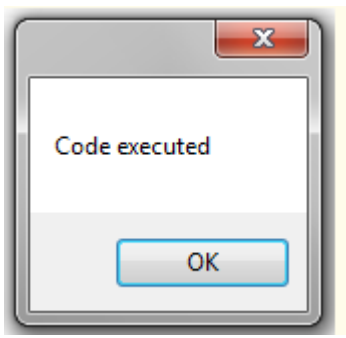
```
void caller()
{
    Message("John", "Hello");
}

void Message(string message, string title)
{
    MessageBox.Show(message, title);
}
```

Same code with return type:

```
void caller()
{
    string s = Message("John", "Hello");
    MessageBox.Show(s);
}

string Message(string message, string title)
{
    MessageBox.Show(message, title);
    return "Code executed";
}
```



Any number of parameters can be sent to the method. The return type can be anything (e.g. string, int, bool, object) but there can only be one.

When you wish to use an instance method, you must instantiate its object as opposed to static methods that can be called any time. The `this` reference refers to its containing object and may be used to refer to its containing object's members, including methods.

Source: [A method is any block of code you plan to...](#) (18 December 2013 1:33 AM)

5.20.1. Function Parameters

In C# when a variable is passed to a function, by default it is done "by value" rather than "by reference". This means a copy of a variable is being sent and changes made in the called function will have no effect on the variable in the calling function.

ref modifier

```
static void Main( string [] args)
{
    int number = 20;
    AddFive(number);
}

static void AddFive( int number)
{
    int Number = number + 5;
}
```

With the code above, number remains 20 in the calling function, but is 25 in the function below.

```
static void Main( string [] args)
{
    int number = 20;
    AddFive(ref number);
}

static void AddFive(ref int number)
{
    int Number = number + 5;
}
```

out modifier

Similar to ref. A value passed via ref must be initialized before the function call. With the out modifier, it need not be initialized in the calling function, but it can't leave the sub function without being assigned a value.

5.20.2. Static Methods

Methods that operate within a class rather than within an object.

They require modifiers:

- public static method

Don't have to create object first.

6. OOP

6.1. Example

```
public partial class FrmBoxTester : Form
{
    public FrmBoxTester()
    {
        InitializeComponent();
    }

    private void BtnTest_Click(object sender, EventArgs e)
    {
        Box b1 = new Box(2.5, 3.0, 5.0);
        MessageBox.Show("Width is " + b1.Wid);
        MessageBox.Show("The volume is " + b1.volume());
    }
}

=====

class Box
{
    //fields
    private double length, width, height;

    //constructors
    public Box(double w, double h, double l)
    {
        width = w;
        height = h;
        length = l;
    }

    //methods
    public double volume()
    {
        return width * height * length;
    }

    public double Wid
    {
        get { return width; }
        set { width = value; }
    }
}
```

7. Collections

7.1. Arrays

Arrays hold a fixed number of objects, all of the same type.
Array size can never be changed.

Declaration

```
int[] Values = new int[4];  
    All four location will be initialize to zero  
int[] arr = new int[3] {7,9,4};  
int[] arr = { 2, 3, 4 };  
  
string[] mystrings = new string[4];  
string[] myStrings = {"one","two","three","four"};
```

Functions

myStrings[0]	Returns "one"
myStrings.GetLength();	Returns 4
Array.Clear(myStrings,0,3)	Starting at 0, clears 3
Array.Sort(myStrings);	
Array.Reverse(myStrings);	

2 dimensional arrays - Tables

```
item[r,c]  
for 3 x 4 table  
declare: int[,] stock = new int[4,5] leaving col 1 empty  
StreamReader read = new StreamReader(f);  
for (int r = 1, r <=3; r++)  
{  
    for (int c = 1; c<=4; c++)  
    {  
        stock [r,c] = convert.ToInt32(read.ReadLine());  
    }  
}
```

7.2. List

A list is the same as an array, except that you don't have to specify the number of values in advance

<https://docs.google.com/document/d/1QiUF4Kw8G3mUumIiqAg0-JqGZ2YXnB6PVR3xqgiY5GM/edit?usp=sharing>
file:\\\\J:\Teach\CSharp\Gemini\C# List Methods Explained.docx

Declaration

```
1. List<string> names = new List<string>();  
2. List<string> names = new List<string>(6);  
3. List<string> names = new List<string>() { "Jenny", "Paul" };
```

```
List<int> numbers = new List<int>();
```

The List is held in contiguous memory. Method 1 seems to have a capacity of 4. When there are 5 additions, capacity generally doubles.

Adding values

```
Names.add("Adam");  
Name[0] then returns Adam
```

```
foreach (string name in names)
{
    listBox1.Items.Add(name);
}
-----
```

List methods

Add	Adds an element at the end of a List<T>.
AddRange	Adds elements of the specified collection
BinarySearch	Search for element and return index
Capacity	Max entries before forced expansion.
Clear	Removes all the elements from a List<T>.
Contains	Checks element existence
Find	Finds the first element
Foreach	Iterates through a List<T>.
Insert	Inserts an element at specified
InsertRange	Inserts elements of another collection
Remove	Removes the first occurrence of element.
RemoveAt	Removes the element at the specified index.
RemoveRange	Removes range of elements
Sort	Sorts all the elements.
TrimExcess	Sets the capacity.
TrueForAll	Determines whether every element is true

7.3. Arraylist

7.4. Stacks

Stacks maintain a list of items that operate on a push-on, pop-off via a LIFO methodology using System.Collections;

Declaration

```
Stack myStack = new Stack();
```

Functions

```
myStack.Push(value);
value = myStack.Pop();
value = myStack.Peek();
x = myStack.Count;
myStack.Clear();
```

7.5. Dictionaries

Dictionaries are basically lookup tables where you use a key to lookup a value
No duplicate keys allowed.

```
using System.Collections;
```

Declaration

```
Hashtable myHT = new Hashtable();
```

Examples

```
myHT.Add("SEA", "Seattle Tacoma Airport");
myHT.Add("SFO", "San Francisco Airport");
myHT["IAD"] = "Washington Dulles Airport";
myHT.Remove("SFO");
```

```
x = myHT.Count;
bool b = myHT.ContainsKey(SFO);
bool b = myHT.ContainsValue("San Fransidco Airport);
```

7.6. Queue

Queues maintain a list of items, but are FIFO (first in first out) using System.Collections;

```
Declaration
Queue myQ = new Queue();
myQ.Enqueue("string 1");
s = myQ.Dequeue();
s = myQ.Peek();
x = myQ.Count;
```

7.7. Char Arrays

```
char[] c = new char[20];
```

```
string s = "John";
char[] c = s.ToCharArray();
c[0]='J'
c[1]='o'
c[2]='h'
c[3]='n'
c[4]='\0'
```

```
StringBuilder sb = new StringBuilder();
```

```
for (int i = 0; i < c.Length; i++)
{
    if (c[i] == '\0') break;
    sb.Append(c[i]);
}
```

```
string t = sb.ToString()
```

Some Char methods
 bool digit = char.IsDigit(value);
 char.IsLetter
 char.IsLetterOrDigit
 char.IsLowerIsUpper
 char.IsWhiteSpace
 char.ToUpper
 char.ToLower
 ToString

```
public static bool IsCharDigit(char c)
{
    return ((c >= '0') && (c <= '9'));
}
```

Source: [char\[\] c = new char\[20\];](#) (18 December 2013 1:40 AM)

7.8. Hashsets

<https://www.youtube.com/watch?v=wHKIP8cov9w>

In C#, a `HashSet<T>` is an optimized, unordered collection of **unique** elements. It is part of the `System.Collections.Generic` namespace and is the go-to collection when you need high-performance set operations and must ensure no duplicates exist.

How a HashSet Works

Under the hood, a `HashSet` uses a **hash table**. When you add an element, C# computes a hash code for it. This allows the collection to check for the existence of an item in near-constant time, regardless of how many items are in the set.

1. Key Properties

The `HashSet<T>` class is lightweight and focuses on performance rather than metadata.

- **Count:** Returns the number of elements currently in the set.
- **Comparer:** Returns the `IEqualityComparer<T>` used to determine equality for the values in the set (useful if you provided a custom rule for comparing objects).

2. Essential Methods

Methods in a `HashSet` are generally $O(1)$ for lookups and additions, making them significantly faster than a `List` for checking "does this exist?"

Basic Operations

- **Add(T item):** Adds an element. Returns true if added, false if the item was already present.
- **Remove(T item):** Removes the specific item. Returns true if found and removed.
- **Contains(T item):** Checks if the item exists. This is extremely fast.
- **Clear():** Removes all elements.

Set Logic (Mathematical Operations)

One of the primary reasons to use `HashSet` is for mathematical set operations:

Method	Description
UnionWith(IEnumerable)	Modifies the set to contain all elements present in itself, the other collection, or both.
IntersectWith(IEnumerable)	Modifies the set to contain only elements present in both collections.
ExceptWith(IEnumerable)	Removes all elements in the specified collection from the current set.
SymmetricExceptWith(...)	Modifies the set to contain only elements that are present in either the set or the collection, but not both .

3. Practical Code Example

Here is how you would typically use a HashSet to manage a list of unique usernames and perform a comparison.

```
C#
using System;
using System.Collections.Generic;

public class Program
{
    public static void Main()
    {
        // 1. Initialization
        HashSet<string> activeUsers = new HashSet<string> { "Alice", "Bob", "Charlie" };

        // 2. Adding (Duplicates are ignored)
        bool wasAdded = activeUsers.Add("Alice"); // Returns false
        activeUsers.Add("Dave");                  // Returns true

        // 3. Fast Lookup
        if (activeUsers.Contains("Bob"))
        {
            Console.WriteLine("Bob is active.");
        }

        // 4. Set Operations
        HashSet<string> bannedUsers = new HashSet<string> { "Charlie", "Eve" };

        // Remove banned users from the active set
        activeUsers.ExceptWith(bannedUsers);

        // 5. Iterating
        foreach (string user in activeUsers)
        {
            Console.WriteLine(user); // Output: Alice, Bob, Dave (order not guaranteed)
        }
    }
}
```

}

4. When to Use HashSet vs. List

Feature	HashSet<T>	List<T>
Duplicates	Not allowed	Allowed
Ordering	Unordered	Maintains insertion order
Lookup Speed	$O(1)$ (Very Fast)	$O(n)$ (Slower as list grows)
Memory	Higher (stores hash codes)	Lower
Access by Index	No	Yes (list[0])

Important Considerations

1. **Reference Types:** If you store custom objects (like a User class), the HashSet uses the object's memory address to check for uniqueness unless you override GetHashCode() and Equals() in your class.
2. **Nulls:** A HashSet can store **one** null value if it is a reference type.
3. **No Sorting:** If you need a set that stays sorted, use SortedSet<T> instead.

Would you like me to show you how to implement GetHashCode and Equals for a custom class so it works correctly with a HashSet?

8. Strings

<https://learn.microsoft.com/en-us/dotnet/api/system.string?view=net-10.0&redirectedfrom=MSDN#methods>

8.1. String Library

8.1.1. Compare

8.1.2. Contains

Returns bool whether a string is contained with a string
the comparison is case sensitive

```
string s = "Test";  
bool x = s.Contains("es");  
returns true
```

8.1.3. CopyTo

Converts string into character array

8.1.4. Format

Allows the placement of variables inside other strings

```
s.Format({0}, "Hello");  
s.Format("There are {0} seconds, {1} minutes in a year", i, j);
```

Console.WriteLine calls format directly
Console.WriteLine("There are {0} seconds, {1} minutes in a year", i, j);

8.1.5. IndexOf

Returns position of first instance of a character or string
case sensitive

Checks if char or string is contain within another string `x = s.IndexOf('@');`
`x = s.IndexOf("Test");`

returns position of first instance of character
returns -1 if the character is not in string

You can also **specify where to start the search** and how far to search
`x = s.IndexOf("@", start, number);`
`x`

8.1.6. IndexOfAny

Returns first position of any of the char arguments

Returns the index position of first instance of any of list of characters or a -1 if none are found.

```
x = s.IndexOf('.', e);
```

8.1.7. Insert

Inserts characters into a string

```
string s="Some Text")
s = s.Insert(5, "More ");
    5 is the position of the insertion
```

8.1.8. Join

Joins elements of an array into a string

```
s.Join(array, delimiter);
```

```
string [] array = {"A", "B", "C"};
string s = string.Join(",", array);
Result: s = "A,B,C";
```

The string can be split with split command

8.1.9. LastIndexOf

Reports index position of the last occurrence of a specified character or string

```
x = LastIndexOf(Char);
x = LastIndexOf(string);
```

Last instances starting at position a

```
x = LastIndexOf(Char, a);
x = LastIndexOf(string, a);
```

Last Instance starting at a in b characters

```
x = LastIndexOf(Char, a, b);
x = LastIndexOf(string, a,b);
```

8.1.10. LastIndexOfAny

Reports index position of the last occurrence of any of specified characters

8.1.11. Length

Returns length of string

```
int x = s.Length;
    Don't need round brackets because length is property not function
```

8.1.12. PadLeft

Pads string to left with specified character

```
s = s.PadLeft(10, '*');
```

8.1.13. PadRight

Pads String to right

8.1.14. Remove

Removes characters from string

```
s.Remove(x);  
Remove chars from x to end
```

```
s.Remove(x,y)  
Removes y chars starting at x
```

```
string s = "0123456789";  
s=s.Remove(5);0,x3  
Result: s="01234"
```

```
string s = "0123456789";  
s=s.Remove(5,3);  
result: s = "0123489"
```



8.1.15. Replace

Replaces one string with another within a string

```
string s = "I made a mistak";  
s = s.Replace("mistak", "mistake");
```

8.1.16. Split

Separates strings into fields of a string array

```
s.Split(delimiter);  
common delimiters  
, comma for csv file  
/n newline  
space for words in sentence
```

—

```
string s = "A,B,C";  
string[] array = s.Split(',');
```

Result:

```
array[0]="A";  
array[1]="B";  
array[2]="C";
```

string can be put back together with 'join' command

8.1.17. StartsWith

8.1.18. Substring

Copies substring from another string

```
s.Substring(start, length);
```

```
string s="abcdefghijklmnopq";  
s = s.Substring(4, 3);  
Result: s = "efg";
```

8.1.19. ToCharArray

Copies the characters to a character array

```
string s = new string(c);
```

8.1.20. ToLower

Converts string to all lower case

8.1.21. ToString

Converts integers, floats, and doubles into string

```
string s1,s2;  
double number = 123.4567;  
s1 = number.ToString();  
s2 = number.ToString("00.00");  
    Formatted number
```

8.1.22. ToUpper

Converts string to all upper case

8.1.23. Trim

Trims start and end of string

```
string s[]= " Testing ";  
s = s.Trim();
```

Result: "Testing"

8.1.24. TrimEnd

Removes chars from end of string

8.1.25. TrimStart

Removes chars from start of string

8.2. String formatters

8.2.1. Currency

```
double value = 12345.6789;
Console.WriteLine(value.ToString("C")); //prints $12,345.68
        Console.WriteLine(value.ToString("C3")); //prints $12,345.679
```

8.2.2. Decimal

```
value = 12345;
Console.WriteLine(value.ToString("D"));
// Displays 12345
Console.WriteLine(value.ToString("D8"));
// Displays 00012345
```

8.2.3. Fixed-Point

```
int numb = 12345;
Console.WriteLine(numb.ToString("F")); //prints 12,345.00
        Console.WriteLine(numb.ToString("F3")); //prints 12,345.000

double numb2 = 12345.6789;
Console.WriteLine(numb2.ToString("F")); //prints 12,345.68
        Console.WriteLine(numb2.ToString("F3")); //prints 12,345.679
```

8.3. StringBuilder

8.3.1. Append

Append string representation of specified type

Each argument will automatically have its ToString method called.

AppendBoolean)
Append(Byte)
Append(Char)
Append(Char[])
Append(Decimal)
Append(Double)
Append(Int16)
Append(Int32)
Append(Int64)
Append(Object)
Append(SByte)
Append(Single)
Append(String)
Append(UInt16)
Append(UInt32)
Append(UInt64)
Append()

Source: [Append string representation of specified type](#) (17 December 2013 11:10 AM)

8.3.2. AppendFormat

Append string representation of specified type

Each argument will automatically have its ToString method called.

AppendBoolean)
Append(Byte)
Append(Char)
Append(Char[])
Append(Decimal)
Append(Double)
Append(Int16)
Append(Int32)
Append(Int64)
Append(Object)
Append(SByte)
Append(Single)
Append(String)
Append(UInt16)
Append(UInt32)
Append(UInt64)
Append()

Source: [Append string representation of specified type](#) (17 December 2013 11:10 AM)

8.3.3. AppendLine

Appends the newline to the end of the current StringBuilder object.

Source: [Appends the newline to the end of the current StringBuilder object.](#) (17 December 2013 11:10 AM)

8.3.4. Clear

Removes all characters from the current StringBuilder instance.

Source: [Removes all characters from the current StringBuilder instance.](#) (17 December 2013 11:11 AM)

8.3.5. CopyTo

Copies the characters from a specified segment of this instance to a specified segment of a destination Char array.

Source: [Copies the characters from a specified segment of this instance...](#) (17 December 2013 11:11 AM)

8.3.6. EnsureCapacity

Ensures that the capacity of this instance of StringBuilder is at least the specified value.

Source: [Ensures that the capacity of this instance of StringBuilder is...](#) (17 December 2013 11:12 AM)

8.3.7. Equals

Determines whether the specified Object is equal to the current Object.

1. Equals(Object);
2. Equals(StringBuilder);

Source: [Determines whether the specified Object is equal to the current Object.](#) (17 December 2013 11:12 AM)

8.3.8. Finalize

Allows an object to try to free resources and perform other cleanup operations before it is reclaimed by garbage collection.

Source: [Allows an object to try to free resources and perform...](#) (17 December 2013 11:12 AM)

8.3.9. GetHashCode

Serves as a hash function for a particular type.

Source: [Serves as a hash function for a particular type.](#) (17 December 2013 11:13 AM)

8.3.10. GetType

Gets the Type of the current instance.

Source: [Gets the Type of the current instance.](#) (17 December 2013 11:13 AM)

8.3.11. Insert

Inserts string representation of type at specified location

```
Insert(Int32, Boolean);
Insert(Int32, Byte);
Insert(Int32, Char);
Insert(Int32, Char[]);
Insert(Int32, Decimal);
Insert(Int32, Double);
Insert(Int32, Int);
Insert(Int32, Int);
Insert(Int32, Object);
Insert(Int32, SByte);
Insert(Int32, Single);
Insert(Int32, String);
Insert(Int32, UInt);
```

Source: [Inserts string representation of type at specified location](#) (17 December 2013 11:14 AM)

8.3.12. MemberwiseClone

Creates a shallow copy of the current Object

Source: [Creates a shallow copy of the current Object](#) (17 December 2013 11:14 AM)

8.3.13. Remove

Removes the specified range of characters from this instance

Source: [Removes the specified range of characters from this instance](#) (17 December 2013 11:14 AM)

8.3.14. Replace

Replaces all occurrences of specified type with another.

```
Replace(Char, Char);
Replace(String, String);
```

Source: [Replaces all occurrences of specified type with another.](#) (17 December 2013 11:15 AM)

8.3.15. ToString

Converts the value of this instance to a String.

Source: [Converts the value of this instance to a String.](#) (17 December 2013 11:15 AM)

9. Files

9.1. Files Library

9.1.1. _____

Moves a file by renaming it.

```
File.Move(source, destination);
```

Will throw except if destination already exists.

Source: [Moves a file by renaming it.](#) (17 December 2013 10:47 AM)

::

9.1.2. AppendAllLines

Appends lines to a file, and then closes the file

Source: [Appends lines to a file, and then closes the file](#) (17 December 2013 10:40 AM)

9.1.3. AppendAllText

Opens a file, appends the specified string to the file, and then closes the file.

If the file does not exist, this method creates a file, writes the specified string to the file, then closes the file.

Mainly useful for log files.

Source: [Opens a file, appends the specified string to the file,...](#) (17 December 2013 10:41 AM)

9.1.4. AppendText

Creates a StreamWriter that appends UTF-8 encoded text to an existing file

Source: [Creates a StreamWriter that appends UTF-8 encoded text to an...](#) (17 December 2013 10:41 AM)

9.1.5. Close

9.1.6. Copy

Copies a file from one location to another leaving both on disk.

1. File.Copy(source, destination);
2. File.Copy(source, destination,true);

Method 1 will throw exception if file already exists at destination. Third parameter forces overwrite.

Source: [Copies a file from one location to another leaving both...](#) (17 December 2013 10:41 AM)

9.1.7. Create

Creates or overwrites a file in the specified path

1. Create(FN);
2. Create(FN, Int32); Specifies buffer size
3. Create(FN, int32,FileOptions);
4. Create(FN, int32,FileOptions,FileSecurity);

Source: [Creates or overwrites a file in the specified path](#) (17 December 2013 10:42 AM)

9.1.8. Decrypt

Decrypts a file that was encrypted by the current account using the Encrypt method.

Source: [Decrypts a file that was encrypted by the current account...](#) (17 December 2013 10:42 AM)

9.1.9. Delete

Deletes file bypassing recycle bin.

File.Delete(FN);

Will throw exception if file is open.

Will not throw exception if file does not exist.

Source: [Deletes file bypassing recycle bin.](#) (17 December 2013 10:43 AM)

9.1.10. Dispose

9.1.11. Encrypt

Encrypts a file so that only the account used to encrypt the file can decrypt it.

Source: [Encrypts a file so that only the account used to...](#) (17 December 2013 10:43 AM)

9.1.12. Open File Dialog

The following code will result in browsing the file system and selecting a file

```
OpenFileDialog ofd = new OpenFileDialog();
ofd.Title = "Select file to be Processed";
ofd.Filter = "Text File|*.txt|CSV|*.csv";
if (ofd.ShowDialog() == System.Windows.Forms.DialogResult.OK)
{
    MessageBox.Show(ofd.FileName);
    button2.Enabled = true;
}
else{
    MessageBox.Show("No File Selected");
}
```

FileName	Complete path to file
SafeFileName	Filename only

To limit to file type, e.g.
ofd.Filter = PNG Image|*.png;

ofd.Title
provides title to dialog

Source: [_ OpenFileDialog ofd = new OpenFileDialog\(\);](#) (17 December 2013 11:20 AM)

9.1.13. Exists

Determines whether file exist and returns bool.

```
bool exists = File.Exists(filename);
```

Source: [Determines whether file exist and returns bool.](#) (17 December 2013 10:44 AM)

9.1.14. File.Replace

Replace content of one file with another.

1. File.Replace(FN1, FN2);
2. File.Replace(FN1, FN2, FN3);
3. File.Replace(FN1, FN2, FN3, true);

FN3 is a backup name for FN1.

```
// Replace contents of file 1 with contents of file 2.
// ... Also create file 3 as backup.
```

```
File.Replace(wfn, rfn, Path.ChangeExtension(rfn, ".bak"));
```



3. Ignores any errors.

Source: [Replace content of one file with another.](#) (17 December 2013 10:50 AM)

9.1.15. GetAccessControl

Gets a FileSecurity object that encapsulates the access control list (ACL) entries for a specified file.

Source: [Gets a FileSecurity object that encapsulates the access control list...](#) (17 December 2013 10:44 AM)

9.1.16. GetAttributes

Gets the FileAttributes of the file on the path.

Source: [Gets the FileAttributes of the file on the path.](#) (17 December 2013 10:44 AM)

9.1.17. GetCreationTimeUtc

Returns the creation date and time of the specified file or directory.

Source: [Returns the creation date and time of the specified file...](#) (17 December 2013 10:45 AM)

9.1.18. GetLastAccessTime

Returns the date and time the specified file or directory was last accessed.

Source: [Returns the date and time the specified file or directory...](#) (17 December 2013 10:46 AM)

9.1.19. GetLastWriteTime

Gets last time a file was written to.

`DateTime d = File.GetLastWriteTime(filename);`

Can also be use without the UTC but it is slower because the latter then calls ToLocalTime.

Source: [Gets last time a file was written to.](#) (17 December 2013 10:46 AM)

9.1.20. Open

Opens a FileStream on the specified path with read/write access.

```
FileStream wp = File.Open(FN, FileMode.Create);
```

FileMode alternatives:

```
FileMode.Append  
FileMode.Create  
FileMode.CreateNew  
FileMode.Open  
FileMode.OpenRead  
FileMode.OpenText  
FileMode.OpenWrite  
FileMode.OpenOrCreate  
FileMode.Truncate
```

Source: [Opens a FileStream on the specified path with read/write access.](#) (17 December 2013 10:47 AM)

9.1.21. OpenRead

Opens an existing file for reading.

Source: [Opens an existing file for reading.](#) (17 December 2013 10:48 AM)

9.1.22. OpenText

Opens an existing UTF-8 encoded text file for reading.

Source: [Opens an existing UTF-8 encoded text file for reading.](#) (17 December 2013 10:48 AM)

9.1.23. OpenWrite

Opens an existing file or creates a new file for writing.

Source: [Opens an existing file or creates a new file for writing.](#) (17 December 2013 10:48 AM)

9.1.24. Peek

9.1.25. Read

9.1.26. ReadAllBytes

Opens a binary file, reads the contents of the file into a byte array, and then closes the file.

Source: [Opens a binary file, reads the contents of the file...](#) (17 December 2013 10:49 AM)

9.1.27. ReadAllLines

Opens a text file, reads all lines of the file, and then closes the file.

1. ReadAllLines(String);
2. ReadAllLines(String, Encoding);

2. Opens a file, reads all lines of the file with the specified encoding, and then closes the file.

Source: [Opens a text file, reads all lines of the file,...](#) (17 December 2013 10:49 AM)

9.1.28. ReadAllText

Opens a text file, reads all lines of the file, and then closes the file.

1. ReadAllText(String);
2. ReadAllText(String, Encoding);

2. Opens a file, reads all lines of the file with the specified encoding, and then closes the file.

Source: [Opens a text file, reads all lines of the file,...](#) (17 December 2013 10:50 AM)

9.1.29. ReadLine

9.1.30. ReadLines

Reads the lines of a file.

1. ReadLines(String);
2. ReadLines(String, Encoding);

Source: [Reads the lines of a file.](#) (17 December 2013 10:50 AM)

9.1.31. SetAccessControl

Applies access control list (ACL) entries described by a FileSecurity object to the specified file.

Source: [Applies access control list \(ACL\) entries described by a FileSecurity...](#) (17 December 2013 10:51 AM)

9.1.32. SetAttributes

Sets the specified FileAttributes of the file on the specified path.

Source: [Sets the specified FileAttributes of the file on the specified path.](#) (17 December 2013 10:51 AM)

9.1.33. SetCreationTime

Sets the date and time the file was created.

1. SetCreationTime
2. SetCreationTimeUtc

Source: [Sets the date and time the file was created.](#) (17 December 2013 10:52 AM)

9.1.34. SetLastAccessTime

Sets the date and time the specified file was last accessed.

1. SetLastAccessTime
2. SetLastAccessTimeUtc

Source: [Sets the date and time the specified file was last accessed.](#) (17 December 2013 10:52 AM)

9.1.35. SetLastWriteTime

Sets the date and time that the specified file was last written to.

1. SetLastWriteTime
2. SetLastWriteTimeUtc

Source: [Sets the date and time that the specified file was...](#) (17 December 2013 10:52 AM)

9.1.36. Write

9.1.37. WriteAllBytes

Creates a new file, writes the specified byte array to the file, and then closes the file. If the target file already exists, it is overwritten.

Source: [Creates a new file, writes the specified byte array to...](#) (17 December 2013 10:53 AM)

9.1.38. WriteAllLines

Creates a new file, writes a collection of strings to the file, and then closes the file

Source: [Creates a new file, writes a collection of strings to...](#) (17 December 2013 10:53 AM)

9.1.39. WriteAllText

Creates a new file, writes the specified string to the file, and then closes the file.

If the target file already exists, it is overwritten

1. WriteAllText(String, String);
2. WriteAllText(String,String,Encoding);

Source: [Creates a new file, writes the specified string to the...](#) (17 December 2013 10:53 AM)

9.2. StreamReader Writer

Use StreamReader to read characters (text) from a stream, such as a file. It's ideal for reading text files line-by-line or in chunks.

Instantiate

```
StreamReader rp = new StreamReader(filePath);  
StreamWriter wp = new StreamWriter(filePath <,true>);  
    Without ,true any file is overwrote  
    With ,true any file is appended
```

Read line by line

```
string line;  
while ((line = rp.ReadLine()) != null)  
{}
```

Write line by line

```
wp.WriteLine(line);  
wp.Write(line);  
Writelines include newline, Write does not
```

Close files

```
rp.Close();  
wp.Close();
```

Error Checks

```
File.Exists( filePath ) == true
```

Common operations

To address a file in user's document directory

```
string fn = "\\testfile";  
filepath = Environment.GetFolderPath(Environment.SpecialFolder.MyDocuments) + fn;
```

```
File.Copy(fileToCopy, newLocation);  
File.Move(fileToMove, fileLocation);  
File.Delete(file_path);
```

10. Path

10.1. ChangeExtension

Changes the extension of a path string

e.g. `File.Replace(wfn, rfn, Path.ChangeExtension(rfn, ".bak"));`

Source: [Changes the extension of a path string](#) (17 December 2013 10:54 AM)

10.2. Combine

Combines an array of strings into a path

Source: [Combines an array of strings into a path](#) (17 December 2013 10:54 AM)

10.3. GetDirectoryName

Returns path of file

Source: [Returns path of file](#) (17 December 2013 10:55 AM)

10.4. GetExtension

Returns of extension of the specified path string

Source: [Returns of extension of the specified path string](#) (17 December 2013 10:55 AM)

10.5. GetFileName

Returns the filename and extension of specified path string

Source: [Returns the filename and extension of specified path string](#) (17 December 2013 10:56 AM)

10.6. GetFileNameWithoutExtension

Returns filename without extension

Source: [Returns filename without extension](#) (17 December 2013 10:56 AM)

10.7. GetFullPath

Returns the absolute path for specified path string

Source: [Returns the absolute path for specified path string](#) (17 December 2013 10:56 AM)

10.8. GetInvalidFileNameChars

Gets an array containing the characters that are not allowed

Source: [Gets an array containing the characters that are not allowed](#) (17 December 2013 10:57 AM)

10.9. GetInvalidPathChars

Gets the root directory information of the specified path

Source: [Gets the root directory information of the specified path](#) (17 December 2013 10:57 AM)

10.10. GetPathRoot

Gets the root directory information of the specified path

Source: [Gets the root directory information of the specified path](#) (17 December 2013 10:58 AM)

10.11. GetRandomFileName

Returns a random folder name or file name

Source: [Returns a random folder name or file name](#) (17 December 2013 10:58 AM)

10.12. GetTempFileName

Creates a uniquely named, zero-byte temporary file on disk and returns the full path of that file

Source: [Creates a uniquely named, zero-byte temporary file on disk and...](#) (17 December 2013 10:59 AM)

10.13. GetTempPath

Returns the path of the current user's temporary folder

Source: [Returns the path of the current user's temporary folder](#) (17 December 2013 10:59 AM)

10.14. HasExtension

Determines whether a path includes a file name extension

Source: [Determines whether a path includes a file name extension](#) (17 December 2013 11:00 AM)

10.14.1. IsPathRooted

Gets a value indicating whether the specified path string contains a root

Source: [Gets a value indicating whether the specified path string contains...](#) (17 December 2013 11:00 AM)

11. Directory Class

```
string path = Directory.GetCurrentDirectory();
```

11.1. Methods

S = String

[http://msdn.microsoft.com/en-us/library/system.io.directory\(v=vs.110\).aspx](http://msdn.microsoft.com/en-us/library/system.io.directory(v=vs.110).aspx)

CreateDirectory(S)

CreateDirectory(S, DirectorySecurity)

Delete(S)

Delete(S, Boolean)

EnumerateDirectories(S)

11.2. CreateDirectory(String)

11.3. Delete(String)

Delete(String)

Deletes an empty directory from a specified path.

Also

Delete(String, Boolean)

Deletes the specified directory and, if indicated, any subdirectories and files in the directory

11.4. EnumerateDirectories(String)

EnumerateDirectories(String)

Returns an enumerable collection of directory names **in** a specified path.

Also:

EnumerateDirectories(String, String)

Returns an enumerable collection of directory names that match a search pattern **in** a specified path.

EnumerateDirectories(String, String, SearchOption)

Returns an enumerable collection of directory names that match a search pattern **in** a specified path, and optionally searches subdirectories.

11.5. EnumerateFiles(String)

EnumerateFiles(String)

Returns an enumerable collection of file names in a specified path

Also:

EnumerateFiles(String, String)

Returns an enumerable collection of file names that match a search pattern in a specified path.

EnumerateFiles(String, String, SearchOption)

Returns an enumerable collection of file names that match a search pattern in a specified path, and optionally searches subdirectories

11.6. EnumerateFileSystemEntries(String)

EnumerateFileSystemEntries(String)

Returns an enumerable collection of file names and directory names in a specified path.

Also:

EnumerateFileSystemEntries(String, String)

Returns an enumerable collection of file names and directory names that match a search pattern in a specified path.

EnumerateFileSystemEntries(String, String, SearchOption)

Returns an enumerable collection of file names and directory names that match a search pattern in a specified path, and optionally searches subdirectories.

11.7. Exists

Exists

Determines whether the given path refers to an existing directory on disk.

11.8. GetAccessControl(String)

GetAccessControl(String)

Gets a DirectorySecurity object that encapsulates the access control list (ACL) entries for a specified directory.

Also:

GetAccessControl(String, AccessControlSections)

Gets a DirectorySecurity object that encapsulates the specified type of access control list (ACL) entries for a specified directory.

11.9. GetCreationTime

GetCreationTime

Gets the creation date and time of a directory.

`GetCreationTimeUtc`

Gets the creation date and time, in Coordinated Universal Time (UTC) format, of a directory.

11.10. GetCurrentDirectory

`GetCurrentDirectory`

Gets the current working directory of the application.

11.11. GetDirectories(String)

`GetDirectories(String)`

Returns the names of subdirectories (including their paths) in the specified directory.

`GetDirectories(String, String)`

Returns the names of subdirectories (including their paths) that match the specified search pattern in the specified directory.

`GetDirectories(String, String, SearchOption)`

Returns the names of the subdirectories (including their paths) that match the specified search pattern in the specified directory, and optionally searches subdirectories.

11.12. GetDirectoryRoot

`GetDirectoryRoot`

Returns the volume information, root information, or both for the specified path.

11.13. GetFiles(String)

`GetFiles(String)`

Returns the names of files (including their paths) in the specified directory.

`GetFiles(String, String)`

Returns the names of files (including their paths) that match the specified search pattern in the specified directory.

`GetFiles(String, String, SearchOption)`

Returns the names of files (including their paths) that match the specified search pattern in the specified directory, using a value to determine whether to search subdirectories

Get files from directory

Method `Directory.GetFiles` returns string array with files names (full paths).

[C#]

```
using System.IO;
```

```
string[] filePaths = Directory.GetFiles(@"c:\MyDir\");  
// returns:  
// "c:\MyDir\my-car.BMP"  
// "c:\MyDir\my-house.jpg"
```

Get files from directory (with specified extension)
You can specify search pattern. You can use wildcard specifiers in the search pattern, e.g. „*.bmp“ to select files with the extension or „a*“ to select files beginning with letter „a“.

```
[C#]  
string[] filePaths = Directory.GetFiles(@"c:\MyDir\", "*.bmp");  
// returns:  
// "c:\MyDir\my-car.BMP"
```

Get files from directory (including all subdirectories)
If you want to search also in subfolders use parameter `SearchOption.AllDirectories`.

```
[C#]  
string[] filePaths = Directory.GetFiles(@"c:\MyDir\", "*.bmp",  
                                         SearchOption.AllDirectories);  
// returns:  
// "c:\MyDir\my-car.BMP"  
// "c:\MyDir\Friends\james.BMP"
```

11.14. GetFileSystemEntries(String)

`GetFileSystemEntries(String)`
Returns the names of all files and subdirectories in a specified path.

`GetFileSystemEntries(String, String)`
Returns an array of file names and directory names that that match a search pattern in a specified path.

`GetFileSystemEntries(String, String, SearchOption)`
Returns an array of all the file names and directory names that match a search pattern in a specified path, and optionally searches subdirectories.

11.15. GetLastAccessTime

`GetLastAccessTime`
Returns the date and time the specified file or directory was last accessed.

`GetLastAccessTimeUtc`
Returns the date and time, in Coordinated Universal Time (UTC) format, that the specified file or directory was last accessed.

11.16. GetLastWriteTime

GetLastWriteTime

Returns the date and time the specified file or directory was last written to.

GetLastWriteTimeUtc

Returns the date and time, in Coordinated Universal Time (UTC) format, that the specified file or directory was last written to.

11.17. GetLogicalDrives

GetLogicalDrives

Retrieves the names of the logical drives on this computer in the form "<drive letter>:\".

11.18. GetParent

GetParent

Retrieves the parent directory of the specified path, including both absolute and relative paths.

11.19. Move

Move

Moves a file or a directory and its contents to a new location.

11.20. SetAccessControl

SetAccessControl

Applies access control list (ACL) entries described by aDirectorySecurity object to the specified directory.

11.21. SetCreationTime

SetCreationTime

Sets the creation date and time for the specified file or directory.

SetCreationTimeUtc

Sets the creation date and time, in Coordinated Universal Time (UTC) format, for the specified file or directory.

11.22. SetCurrentDirectory

SetCurrentDirectory

Sets the application's current working directory to the specified directory.

11.23. SetLastAccessTime

SetLastAccessTime

Sets the date and time the specified file or directory was last accessed.

SetLastAccessTimeUtc

Sets the date and time, in Coordinated Universal Time (UTC) format, that the specified file or directory was last accessed.

11.24. SetLastWriteTime

SetLastWriteTime

Sets the date and time a directory was last written to.

SetLastWriteTimeUtcSets the date and time, in Coordinated Universal Time (UTC) format, that a directory was last written to.

11.25. New Note 268

12. Controls

A control is anything the user can interact with.

A control can be added by:

1. Dragging.
2. Click on control, then click on form location.

Controls can be sized or moved on the form.

The lines that appear make it easy to line up with other controls. If you want to move control without lines, hold down the alt key.

Each Control has sets of

1. Properties

2. Events

Menus of each appear on right. Either can be selected via icons at top.

[Controls - Microsoft Learn](#)

Control Prefixes

Check box	Chk
Combo box	Cbo
Button	Btn
Form	Frm
Group Box	Grp
Horizontal scroll bar	Hsb
Label	Lbl
List box	Lst
Menu	Mnu
Picture box	Pic
Radio button	Rdo
Text box	Txt
Vertical scroll bar	Vsb

12.1. Forms

12.1.1. Setup

<https://gemini.google.com/app/920aaddf14114698>

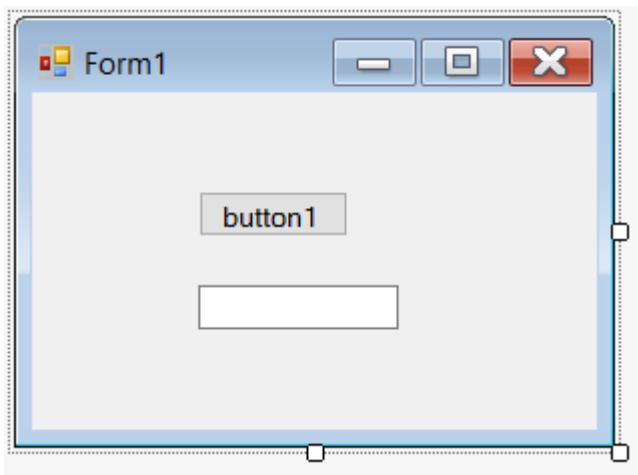
To setup a second window or form.

1. A new form can be initiated either via menu project
2. form2 is then declared on form1 so it can be addressed:
`Form2 f = new Form2(textBox1.Text);`
In this constructor, any number of parameters can be sent.
3. Form 2 can be addressed by either
`f.Show();`
`f.ShowDialog();`
This hands control to second form.

Form1.cs

namespace Tutor

```
{  
    public partial class Form1 : Form  
    {  
        public Form1()  
        {  
            InitializeComponent();  
        }  
  
        private void button1_Click(object sender, EventArgs e)  
        {  
            Form2 f = new Form2(textBox1.Text);  
            // f.Show();  
            f.ShowDialog(); // With this you are now working on form2  
        }  
    }  
}
```



Form2.cs

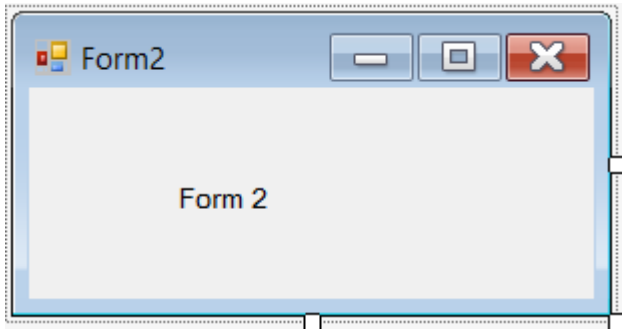
namespace Tutor

```
{  
    public partial class Form1 : Form  
    {  
        public Form1()  
        {  
            InitializeComponent();  
        }  
    }  
}
```

```

private void button1_Click(object sender, EventArgs e)
{
    Form2 f = new Form2(textBox1.Text);
    f.ShowDialog(); // With this you are now working on form2
}
}
}

```



12.1.2. Form Close

```
this.Close();
```

12.1.3. Properties

12.1.3.1. Cursor

Description: Determines the cursor shape displayed when the mouse pointer is over the form or its controls.

Effect of Changes:

Changing Cursor to a value from the Cursors class (e.g., Cursors.Hand, Cursors.WaitCursor, Cursors.Cross) alters the cursor's appearance. For example:

12.1.3.2. StartPosition

Can be set to CenterScreen, CenterParent, Windows default

12.1.3.3. Text

Description: Sets the text displayed in the form's title bar.

Effect of Changes:

1. Modifies the title bar text, which identifies the form to the user.
2. Setting Text = "My Application"; updates the title bar to display "My Application."
3. If set to an empty string (""), the title bar shows no text.
4. Affects appearance and user recognition of the form's purpose.

12.1.3.4. Enabled

if set to false every control is disabled.

Every control has an enable so controls can be selectively disabled.

12.1.3.5. Size, ClientSize

Size and ClientSize

Description: Size sets the total dimensions of the form (including borders and title bar), while ClientSize sets the dimensions of the client area (excluding borders and title bar).

Effect of Changes:

- Changing Size (e.g., `form.Size = new Size(800, 600);`) resizes the entire form, affecting its visual footprint.
- Changing ClientSize adjusts the usable area for controls, indirectly resizing the form to accommodate the specified client area plus non-client elements (borders, title bar).
- Impacts layout and usability, as controls may need repositioning or resizing.

12.1.3.6. Location

Location

Description: Specifies the form's position on the screen (top-left corner coordinates).

Effect of Changes:

- Setting `Location = new Point(100, 100);` positions the form 100 pixels from the top and left of the screen.
- Affects where the form appears when shown, impacting user accessibility and visibility.
- If set to invalid coordinates (e.g., off-screen), the form may not be visible.

12.1.3.7. FormBorderStyle

FormBorderStyle

Description: Controls the appearance and behavior of the form's border.

Effect of Changes:

- Options include `FixedSingle`, `Sizable`, `FixedDialog`, `None`, etc.
 - `FixedSingle`: Fixed border, non-resizable.
 - `Sizable`: Resizable border with minimize/maximize buttons.
 - `None`: No border or title bar, creating a borderless window.
- Affects both appearance (border visibility) and functionality (resizing, minimizing, or maximizing capabilities).
- Example: `FormBorderStyle = FormBorderStyle.None;` creates a borderless form, removing the title bar and borders, useful for custom UI designs

12.1.3.8. WindowState

WindowState

Description: Determines the initial state of the form (normal, maximized, or minimized).

Effect of Changes:

- `Normal`: Displays the form at its default size.
- `Maximized`: Fills the screen.
- `Minimized`: Starts in the taskbar.
- Affects the initial presentation and user interaction with the form

12.1.3.9. BackColor, BackgroundImage

BackColor and BackgroundImage

Description: `BackColor` sets the background color; `BackgroundImage` sets an image as the background.

Effect of Changes:

- BackColor = Color.Blue; changes the form's background to blue, affecting visual style.
- BackgroundImage = Image.FromFile("image.jpg"); displays an image as the background, potentially stretched or tiled based on BackgroundImageLayout.
- Impacts the form's aesthetic and readability of controls.

12.1.3.10. Opacity**Opacity**

Description: Controls the transparency of the form (0.0 to 1.0).

Effect of Changes:

- Opacity = 0.5; makes the form 50% transparent, allowing the desktop or other windows to show through.
- Affects appearance, useful for effects like fade-in/out or semi-transparent overlays.
- Can reduce visibility of controls if too low

12.1.3.11. TopMost**TopMost**

Description: Determines if the form stays on top of other windows.

Effect of Changes:

- TopMost = true; keeps the form above all other non-topmost windows, even when not focused.
- Affects user interaction by ensuring the form remains visible, useful for alerts or dialogs.

12.1.3.12. ControlBox, MinimizeBox, MaximizeBox**ControlBox, MinimizeBox, MaximizeBox**

Description: Control the visibility and functionality of the title bar buttons (close, minimize, maximize).

Effect of Changes:

- ControlBox = false; hides all title bar buttons.
- MinimizeBox = false; disables the minimize button.
- MaximizeBox = false; disables the maximize button.
- Affects user interaction by limiting window controls, often used for dialogs or kiosks

12.1.3.13. Icon**Icon**

Description: Sets the icon displayed in the title bar and taskbar. Also set properties such as size

New icons can be downloaded from: iconfinders.com

12.1.3.14. ShowIcon

Defaults true. If set false no icon is shown upper left.

ShowInTaskbar

Defaults true.

12.1.3.15. StartPosition

StartPosition

Description: Determines where the form appears when first shown.

Effect of Changes:

- Options include CenterScreen, CenterParent, Manual, etc.
- StartPosition = FormStartPosition.CenterScreen; centers the form on the screen.
- Affects initial placement, enhancing user experience.

12.1.3.16. Enabled

Enabled

Description: Enables or disables user interaction with the form.

Effect of Changes:

- Enabled = false; disables all controls on the form, preventing interaction.
- Affects functionality, often used for modal dialogs or during processing.

12.1.3.17. ShowInTaskbar

ShowInTaskbar

Description: Controls whether the form appears in the taskbar.

Effect of Changes:

- ShowInTaskbar = false; hides the form from the taskbar, useful for secondary windows or dialogs.
- Affects visibility in the taskbar, impacting user navigation.

12.1.3.18. TransparencyKey

TransparencyKey

Description: Specifies a color that becomes transparent, allowing the background to show through.

Effect of Changes:

- TransparencyKey = Color.Lime; makes all lime-colored areas transparent, creating shaped forms.
- Affects appearance, often used for non-rectangular windows.

12.1.3.19. WindowState

Controls initial presentation of app.
Can be set to normal, minimise, maximais

12.1.4. Multiple Forms

12.1.4.1. Add new form

A second form is also a second class and follows all of the conventions of a class.

To install a new form:

1. Go to project on menu and click on Add Windows Form.
2. Make sure Windows Form is selected
3. Name the form whatever
4. Click on Add

In Solution explorer, both the original and the new form are displayed.

A design form is presented on which any control can be installed i.e. buttons, labels etc.

However, at this stage form2 can not be addressed from form1.

12.1.4.2. Address Form2

To address form2 we must add an instance of it in form 1, like so.

```
Form2 f = new Form2();
```

The form can then be displayed in two ways via two methods in form class..

```
f.Show();
```

```
f.ShowDialog();
```

The difference is Show will allow you to go back and forth between the two forms while ShowDialog will pass control to form2 and you can't go back and forth.

=====

Variables can be passed from form1 to form 2 via the constructor i.e:

Form1:

```
Form2 f = new Form2(mystring, myint);
```

Form 2:

```
public Form2(string mystring, int myint)
{
    InitializeComponent();
}
```

12.1.4.3. Passing Info

Passing info between forms can be done with get and set, e.g.

Form1

```
public string newname { get; set; }
private void button1_Click(object sender, EventArgs e)
{
    Form2 f = new Form2();
    textBox1.Text = f.s;
}
```

Form 2

```
public partial class Form2 : Form
{
    public string s { get; set; }
    public Form2()
    {
        InitializeComponent();
        s = "This is how to do it";
    }
}
```

12.2. Button



Changing events

Events such as press down, press up can be changed by clicking on **lightning icion** in toolbox. For example, the following code was produced by double clicking MouseHover event.

```
private void Button1_MouseHover(object sender, EventArgs e)
{
    MessageBox.Show("Hovering");
}
```

Changing Button Properties

In addition to property panel changes, properties can also be changed via code, e.g.

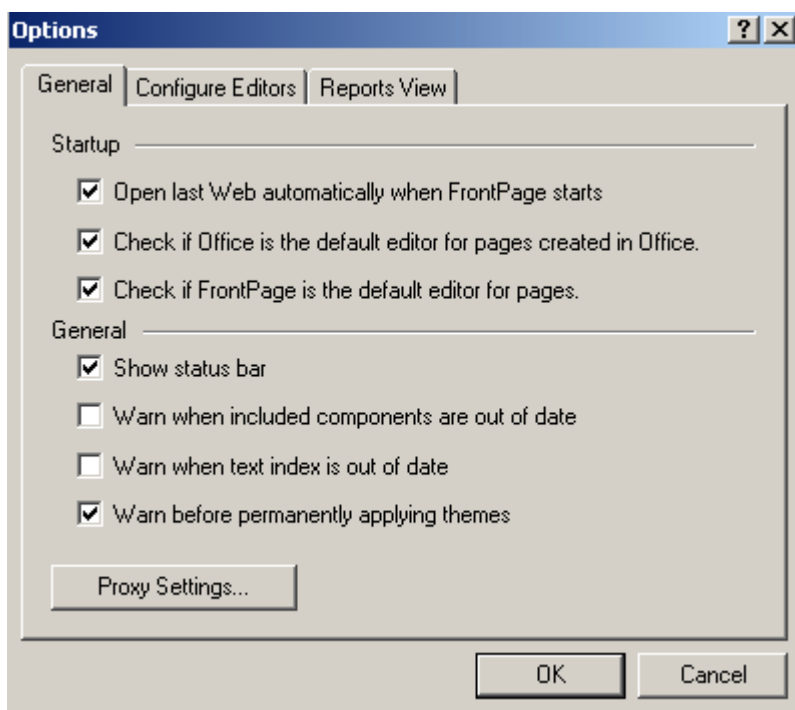
```
button1.Text = "John";
button1.BackColor = Color.Red;
```

Deactivating Button

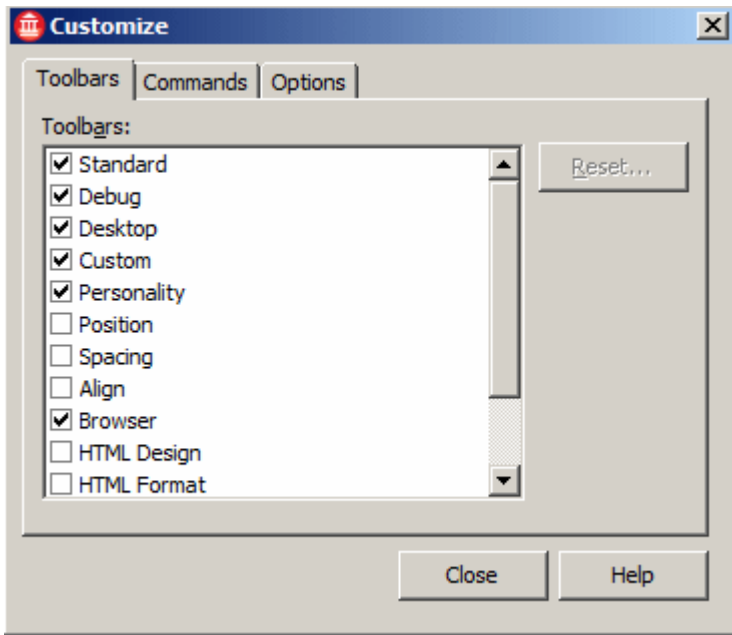
This code will grey out and deactivate button

```
button1.Enabled = false;
```

12.3. Checkbox



12.4. ChekedListBox



12.5. ColorDialog

12.6. ComboBox

[ComboBox in C# - GeeksforGeeks](http://www.geeksforgeeks.com/combobox-in-c/)

A **ComboBox** in C# Windows Forms is a versatile control that combines a dropdown list with an editable text field, allowing users to either select from a predefined list of items or enter custom text. It's commonly used in forms to provide a compact way to present multiple options while saving space compared to other controls like **ListBox** or **RadioButton** groups. Below, I'll explain when and how it's used, along with its major properties.

Items can be entered via property "item". Clicking it will presented a box where items can be entered. It can also be done in code during initialization as follows:

```
combobox1.items.add("New item");
```

to get what has be selected:

```
MessageBox.Show($"You picked {comboBox1.Text}");
```

When to Use a ComboBox

The **ComboBox** is ideal in the following scenarios:

1. **Limited Space:** When you need to present multiple options in a compact UI, such as selecting a country, state, or category.
2. **Predefined Options:** When users need to choose from a fixed set of options (e.g., a list of departments, colors, or statuses).
3. **Editable Input:** When you want to allow users to either select from a list or type a custom value (depending on the **ComboBox** style).
4. **Data Filtering:** When you need to filter or autocomplete options dynamically based on user input (e.g., search suggestions).
5. **Form Data Entry:** In forms for settings, preferences, or data entry where users need to pick from a list or provide custom input.

Examples of Use Cases:

- Selecting a font size in a text editor.
 - Choosing a product category in an e-commerce application.
 - Picking a date format or time zone in a configuration form.
-

How to Use a ComboBox in C# Windows Forms

You can add a ComboBox to a Windows Forms application either via the **Visual Studio Designer** or programmatically in code. Below are the steps for both approaches:

1. Using the Visual Studio Designer

- Open the Windows Forms Designer in Visual Studio.
- Drag a ComboBox control from the Toolbox onto the form.
- Configure its properties in the **Properties** window (e.g., set items, style, or event handlers).
- Add items to the ComboBox:
 - In the Properties window, click the Items property, and enter items in the String Collection Editor.
 - Alternatively, set items programmatically in code.

2. Programmatically Adding a ComboBox

<https://www.extremetech.com/extreme/259977-software-increasingly-complex-thats-dangerous>

Major Properties of ComboBox

The ComboBox control has several key properties that control its behavior and appearance. Here are the most important ones:

1. **Items:**

- Type: `ObjectCollection`
- Description: A collection of items displayed in the dropdown list.
- Example: `comboBox.Items.Add("Item 1");` or `comboBox.Items.AddRange(new string[] { "Item 1", "Item 2" });`

4. **DropDownStyle:**

- Type: `ComboBoxStyle` (enum)
- Description: Controls the behavior of the ComboBox.
- Values:
 - `DropDown` (default): Allows users to select from the list or type a custom value.
 - `DropDownList`: Users can only select from the list (no custom input).
 - `Simple`: Displays a listbox-like view without a dropdown, with an editable text field.
- Example: `comboBox.DropDownStyle = ComboBoxStyle.DropDownList;`

4. **SelectedIndex:**

- Type: `int`
- Description: Gets or sets the index of the currently selected item (-1 if none is selected).
- Example: `comboBox.SelectedIndex = 0;` (selects the first item)

4. **SelectedItem:**

- Type: `object`
- Description: Gets or sets the currently selected item (null if none).
- Example: `var item = comboBox.SelectedItem;`

4. **Text:**

- Type: `string`
- Description: Gets or sets the text in the ComboBox's text field (useful for editable ComboBoxes).
- Example: `comboBox.Text = "Default Text";`

4. **DataSource:**

- Type: `object`
- Description: Binds the ComboBox to a data source (e.g., a List, DataTable, or database query).
- Example:

1. **DisplayMember** and **ValueMember**:

- Type: string
- Description: Used with data-bound ComboBoxes to specify which property of the data source is displayed (DisplayMember) and which is used as the underlying value (ValueMember).

3. **DropDownWidth**:

- Type: int
- Description: Sets the width of the dropdown list.
- Example: `comboBox.DropDownWidth = 200;`

4. **MaxDropDownItems**:

- Type: int
- Description: Sets the maximum number of items visible in the dropdown before a scrollbar appears.
- Example: `comboBox.MaxDropDownItems = 5;`

4. **AutoCompleteMode** and **AutoCompleteSource**:

- Type: `AutoCompleteMode` (e.g., `Suggest`, `Append`, `SuggestAppend`) and `AutoCompleteSource`
- Description: Enables autocomplete functionality by suggesting or completing text as the user types.

3. **Enabled**:

- Type: bool
- Description: Enables or disables the ComboBox.
- Example: `comboBox.Enabled = false;`

4. **Visible**:

- Type: bool
- Description: Shows or hides the ComboBox.
- Example: `comboBox.Visible = true;`

4. **Location** and **Size**:

- Type: `Point` and `Size`
 - Description: Sets the position and size of the ComboBox on the form.
 - Example: `comboBox.Location = new Point(20, 20); comboBox.Size = new Size(200, 25);`
-

Common Events

- **SelectedIndexChanged**: Fired when the selected item changes.
 - Example: `comboBox.SelectedIndexChanged += (s, e) => MessageBox.Show(comboBox.SelectedItem.ToString());`
 - **TextChanged**: Fired when the text in the ComboBox changes (useful for editable ComboBoxes).
 - **DropDown** and **DropDownClosed**: Fired when the dropdown list is opened or closed.
-

Best Practices

- **Use DropDownList for Restricted Input**: Set `DropDownStyle` to `DropDownList` when users should only select from predefined options to prevent invalid input.
 - **Bind to Data Sources**: Use `DataSource`, `DisplayMember`, and `ValueMember` for dynamic data from databases or collections.
 - **Enable Autocomplete**: Use `AutoCompleteMode` and `AutoCompleteSource` to improve user experience in large lists.
 - **Validate Input**: For editable ComboBoxes, validate user input in the `TextChanged` or `Validating` event to ensure it meets requirements.
 - **Keep Lists Manageable**: Avoid overwhelming users with too many items; consider filtering or grouping options if the list is large.
-

If you have a specific use case or need a detailed example (e.g., binding to a database, handling events, or creating a custom ComboBox), let me know, and I can provide more tailored guidance!

Source: [ComboBox Usage in C Forms - Grok and 1 more page - Personal](#) (5 July 2025 1:56 PM)

12.7. ContextMenuStrip

12.8. DataGridView

12.9. DateTimePicker

12.10. DomainUpDown

12.11. ErrorProvider

12.12. FileSystemWatcher

12.13. FlowLayoutPanel

12.14. FolderBrowserDialog

12.15. FontDialog

12.16. GroupBox

12.17. HelpProvider

12.18. HScrollBar

12.19. ImageList

12.20. Label

Properties

Text
Font
ForeColor
BackColor
Autosize
BorderStyle
TextAlign
Visible
Image
Anchor
Dock
FlatStyle

12.21. LinkLabel

12.22. ListBox

<http://csharp.net-informations.com/gui/cs-listbox.htm>

[https://msdn.microsoft.com/en-us/library/system.windows.forms.listbox\(v=vs.110\).aspx](https://msdn.microsoft.com/en-us/library/system.windows.forms.listbox(v=vs.110).aspx)

A C# ListBox control provides a user interface to display a list of items. Users can select one or more items from the list. A ListBox may be used to display multiple columns and these columns may have images and other controls.

The items may exceed the space allotted. If that is the case scrollbars are automatically added, both vertical and horizontal.

The ComboBox provides similar functionality, but all items are shown on one line with a drop down arrow on the right.

Items within the listbox can either be added at design time (via items property) or during program operation.

At **design time**, to create a list of items, access the Properties window of the list box and click the ellipsis button of the Items field. This would open the String Collection Editor where items can be entered one on each line.

Programatic entry

```
listBox1.Items.Add("one");
```

You can also first create an array of items and then add that array to the collection. To support this, use the **AddRange()** method.

items can be removed

```
items.RemoveAt(selectedIndex);
```

```
int index = listBox1.FindString("target");
```

Source: [ListBox in C# and 2 more pages - Personal](#) (13 September 2025 3:51 AM)

Item Selection

The SelectionMode property defines how items are selected in a ListBox. The SelectionMode value can be one of the following four SelectionMode enumeration values:

- *None*: No item can be selected.
- *One*: Only one item can be selected.
- *MultiSimple*: Multiple items can be selected.
- *MultiExtended*: Multiple items can be selected, and the user can use the SHIFT, CTRL, and arrow keys to make selections.

Default is One.

To select an item in a ListBox, we can use the SetSelect method that takes an item index and a true or false value where the true value represents the item to be selected.

If user click on an item for selection:

ListBox.SelectedIndex

ListBox.SelectedItem

By default, the user can select only one item in the list. If you want the user to be able to select more than one item, change the value of the **SelectionMode** property. This property is based on the **SelectionMode** enumeration. After the user has selected more than one item, to get the indexes of the items that are selected, you can access the **ListBox.SelectedIndices** property which holds that list.

Source: [Windows Controls: The List Box and 2 more pages - Personal](#) (14 September 2025 5:13 AM)

We can clear all selected items by calling the ClearSelected method, as in:

```
1. listBox1.ClearSelected();
```

Multicolumn Mode

In C# Windows Forms, the `ListBox` control can display items in a multicolumn format, which is useful for creating snaking, newspaper-style columns from a single list of items. This is different from a `ListView` or `DataGridView`, which displays distinct data points in separate columns.

To use a `ListBox` in multicolumn mode, you need to configure two main properties:

1. **MultiColumn Property:** This boolean property must be set to `true` to enable the multicolumn feature. When `true`, the `ListBox` will arrange its items in columns that are read from top to bottom, then left to right.
2. **ColumnWidth Property:** This integer property sets the width of each column in pixels. You must set this to a value greater than zero for the columns to be rendered correctly.

When multicolumn mode is active, the `ListBox` populates the first column vertically until it's full, then begins a new column to the right and continues filling it. A horizontal scrollbar will appear automatically if the total width of the columns exceeds the width of the control.

How to Implement MultiColumn Mode

Here is how you would typically set up a multicolumn `ListBox` in your code:

```
C#
// Assuming you have a ListBox control named 'myListBox' on your form.
public void SetupMultiColumnListBox()
{
    // 1. Enable the multicolumn feature.
    myListBox.MultiColumn = true; [3, 1]
    // 2. Set the width for each column (e.g., 100 pixels).
    myListBox.ColumnWidth = 100; [2]
    // Now, add items as you normally would.
    // The ListBox will automatically arrange them into columns.
    for (int i = 1; i <= 30; i++)
    {
        myListBox.Items.Add($"Item {i}");
    }
}
```

This setup is ideal for displaying a long list of single data points in a more compact and readable format, preventing excessive vertical scrolling.

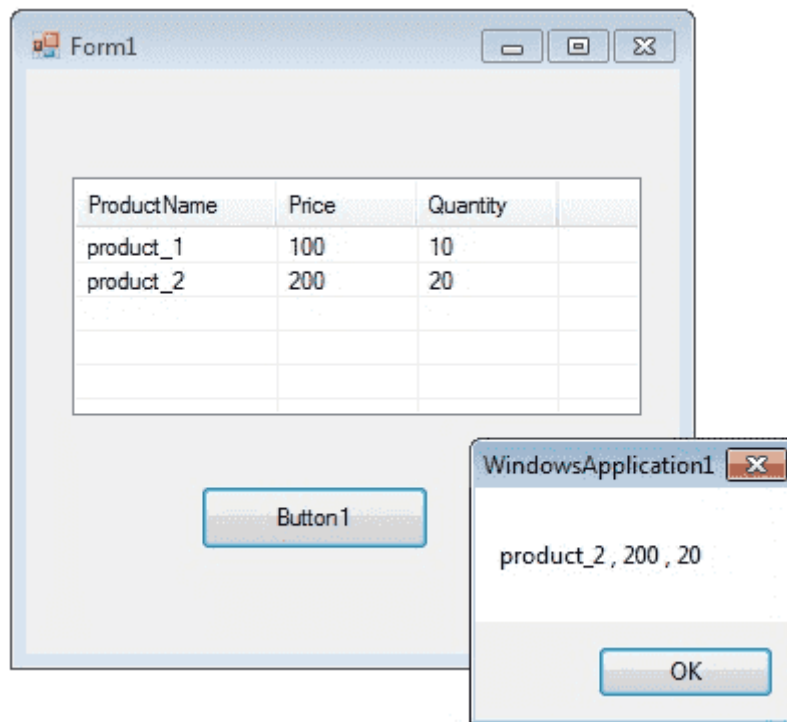
Sources and related content

[Getting Started in Windows Forms MultiColumn ListBox \(GridListControl\)](#)



help.syncfusion.com

12.23. ListView



Add Columns in ListView

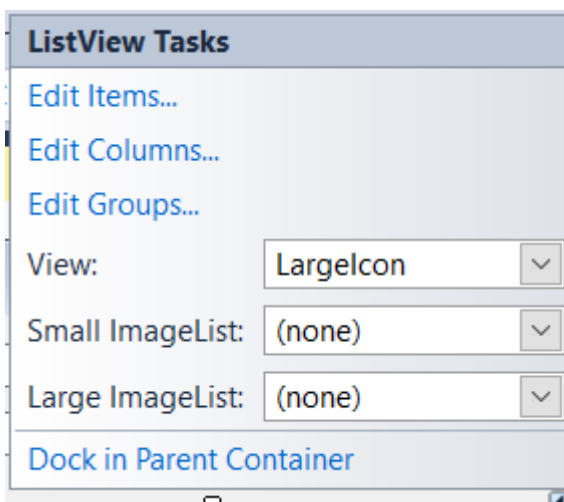
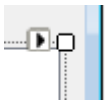
You can add columns in ListView by using Columns.Add() method. This method takes two arguments, first one is the Column heading and second one the column width.

```
listView1.Columns.Add("ProductName", 100);
```

In the above code, "ProductName" is column heading and 100 is column width.

Alternatively

In upper right corner of the inserted list view is a triangle that can be clicked resulting in menu below. One option is to edit columns.



After clicking 'Edit Columns', you can then enter column headers in text property. You will also want to change the view to 'Details' instead of 'LargeIcon'. And you can also resize the columns widths.

Interesting properties

Gridlines

FullRowSelect - If set true, will highlight entire selected row.
Multiselect - If set to false, prevents duplicated entries.
ContextMenuStrip - Message displayed when you right click. Is draged in from toolbox.
Checkbox - if true, adds checkbox at start of each row

Adding entries to ListView - example

```
ListViewItem lvi = new ListViewItem(textBox1.Text);  
lvi.SubItems.Add(textBox2.Text);  
lvi.SubItems.Add(textBox3.Text);  
listView1.Items.Add(lvi);  
textBox1.Text = "";  
textBox2.Text = "";  
textBox3.Text = "";
```

Source: [\[#\] C# ListView Control](#) (24 August 2015 3:26 AM)

12.23.1. Links

<http://www.c-sharpcorner.com/uploadfile/mahesh/working-with-listview-in-C-Sharp/>

<https://www.youtube.com/user/iTzAdam5X>

12.24. MaskedTextBox

12.25. MenuStrip

12.26. MessageBox

[From Perplexity](#)



Show Text or Number in MessageBox

```
MessageBox.Show("Message")
MessageBox.Show(Integer.ToString() );
```

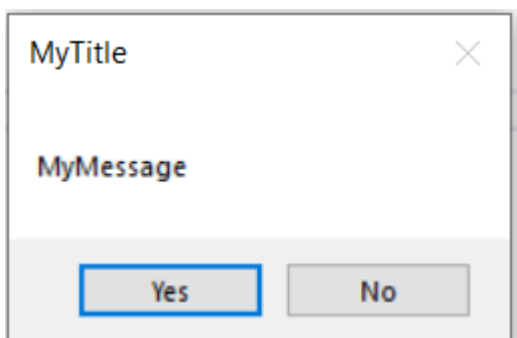
Arguments to MessageBox

1. Message to be displayed
 2. Text in title bar.
 3. Buttons to be display **MessageBoxButtons**.
 - OK
 - OKCancel
 - YesNo
 - YesNoCancel
 - RetryCancel
 - AboutRetryIgnore
 4. Icon to be displayed **MessageBoxIcon**.
 - Asterisk
 - Error
 - Exclamation
 - Hand
 - Information
 - None
 - Question
 - Stop
 - Warning
- MessageBoxDefaultButton.Button2,
 MessageBoxOptions.RightAlign

e.g.

```
DialogResult result = MessageBox.Show(
    "Are you sure you want to exit?",
    "Confirm Exit",
    MessageBoxButtons.YesNoCancel,
    MessageBoxIcon.Question,
    MessageBoxDefaultButton.Button2,
    MessageBoxOptions.RightAlign
);
```

```
MessageBox.Show("MyMessage", "MyTitle", MessageBoxButtons.YesNo);
```



How to handle return from messagebox

The MessageBox.Show method returns a value of type DialogResult, which indicates which button the user clicked

Button	Returns
OK	DialogResult.OK
Cancel	DialogResult.Cancel
Yes	DialogResult.Yes
No	DialogResult.No
Retry	DialogResult.Retry
Abort	DialogResult.Abort
Ignore	DialogResult.Ignore

Return can be handled with if or switch

```
DialogResult result = MessageBox.Show
```

```
if (result == DialogResult.Yes) {}
```

```
switch (result) {  
    case DialogResult.Abort:  
}
```

Source: [What are the options for using MessageBoxes in CSharp and 2 more pages - Personal](#) (18 June 2025 4:57 AM)

Source: [if \(result == DialogResult.Yes\)](#) (18 June 2025 4:56 AM)

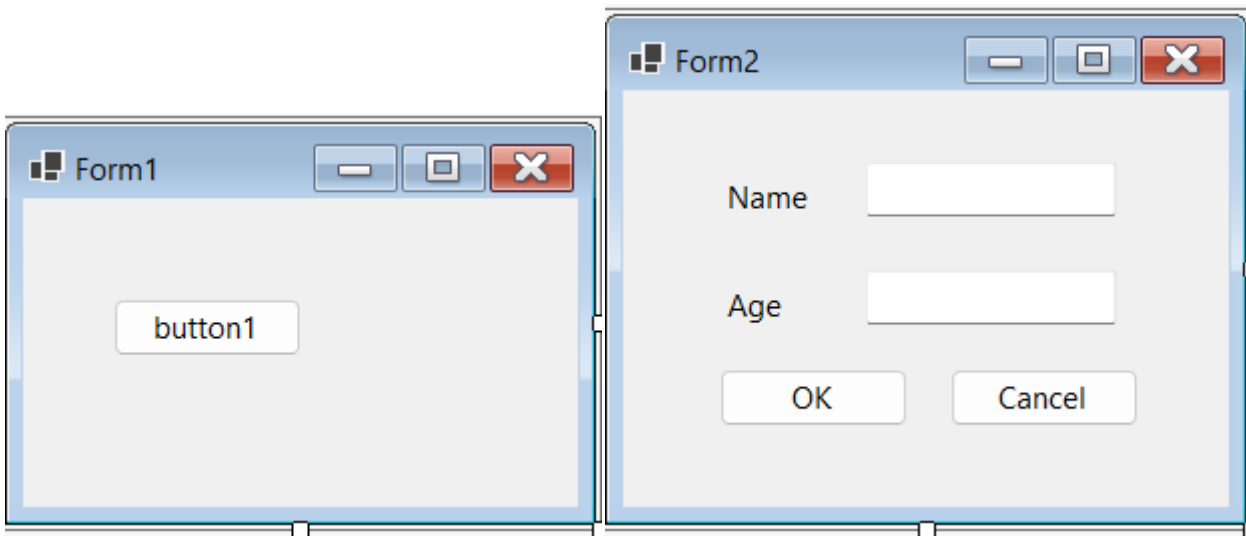
Source: [DialogResult result = MessageBox.Show](#) (18 June 2025 4:56 AM)

Source: [Show Text or Number in MessageBox](#) (17 December 2013 11:39 AM)

12.26.1. Y/N Program

```
private void button1_Click(object sender, EventArgs e)  
{  
    DialogResult r = MessageBox.Show("Do you understand",  
        "Dialog", MessageBoxButtons.YesNo,  
        MessageBoxIcon.Question);  
  
    if (r == DialogResult.Yes)  
    {  
        MessageBox.Show("You pressed yes");  
    }  
}
```

12.26.2. Dialog Box



Form 1 Code

```
Form2 f = new Form2();
DialogResult r = f.ShowDialog();

if (r == DialogResult.OK)
{
    MessageBox.Show($"{f.UserName} is {f.UserAge}");
}

if (r == DialogResult.Cancel)
{
    MessageBox.Show("You pressed cancel");
}
```

Form2 Code

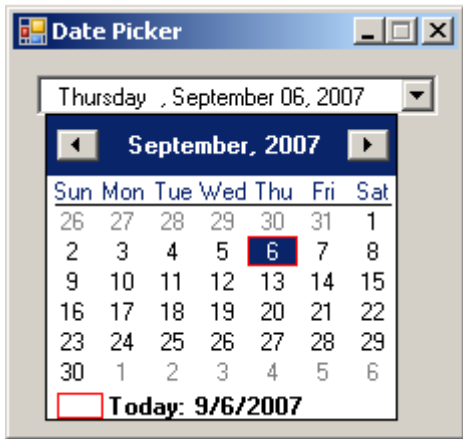
```
public string UserName
{
    get { return TxtName.Text; }
}

public string UserAge
{
    get { return TxtAge.Text; }
}
```

Program Steps

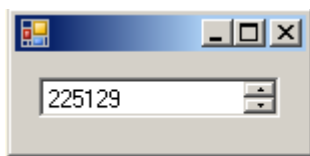
1. Make second form
 - right click program name
 - add
 - add new idem
 - form
2. Make form2 reachable via form1
3. Design Form2 fields
4. Specify button properties
 - Dialog Result
 - OK
 - Cancel
5. Right click form and select view code
6. Enter get statements
7. Process Dialog Results in form1

12.27. MonthCalendar



12.28. NotifyIcon

12.29. NumericUpDown

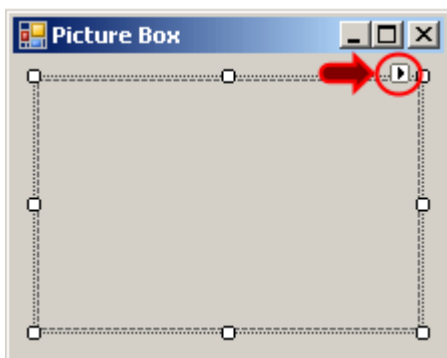


12.30. OpenFileDialog

12.31. PageSetupDialog

12.32. Panel

12.33. PictureBox



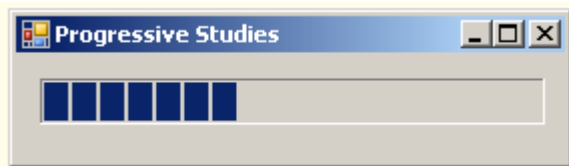
12.34. ProgressDialog

12.35. PrintDocument

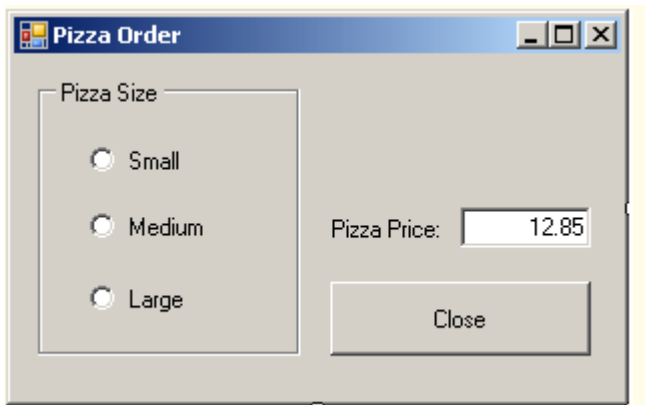
12.36. PrintPreviewControl

12.37. PrintPreviewDialog

12.38. ProgressBar



12.39. RadioButton



When there is a series of radio buttons, when you click one Others are turned off. No two buttons can be on at the same time.

To prevent this, the radio buttons can be grouped as follows

- Draw the frame first
- Next click once on the radio button from the toolbar
- Finally move the mouse inside the frame and draw the radio button

12.40. RichTextBox

https://grok.com/share/c2hhcmQtMg%3D%3D_cf37abfb-3e98-4ccd-a909-99f1243a28ee

12.41. SaveFileDialog

12.42. SplitContainer

12.43. Splitter

12.44. StatusStrip

12.45. TabControl

12.46. TableLayoutPanel

12.47. TextBox

Text boxes are used to display information or to get information from the user. Is often named after the adjacent label.

To programatically put text into a textBox
`textBox1.Text = s;`

To fetch text enter on a textbox
`s = textBox1.Text;`

To fetch a number entered into a text box
`x = int.Parse(textBox.Text);`
`x = float.Parse(textBox.Text);`
`x = double.Parse(textBox.Text);`

To check for empty box
`x = int.TryParse(textBox.Text);`

To detect a specific key e..g. enter

```
private void textBox1_KeyDown(object sender, KeyEventArgs e)  
{  
    if (e.KeyCode == Keys.Enter)  
    {  
        AcceptMethod();  
    }  
}
```

Textbox controls

Fonts
ForeColor
BackColor
BorderStyle
Enable
Multiline
ScrollBars
WordWrap
Password
Max Length
Read Only

12.48. Timer

12.49. ToolStrip

12.50. ToolStripContainer

12.51. ToolTip

12.52. TrackBar

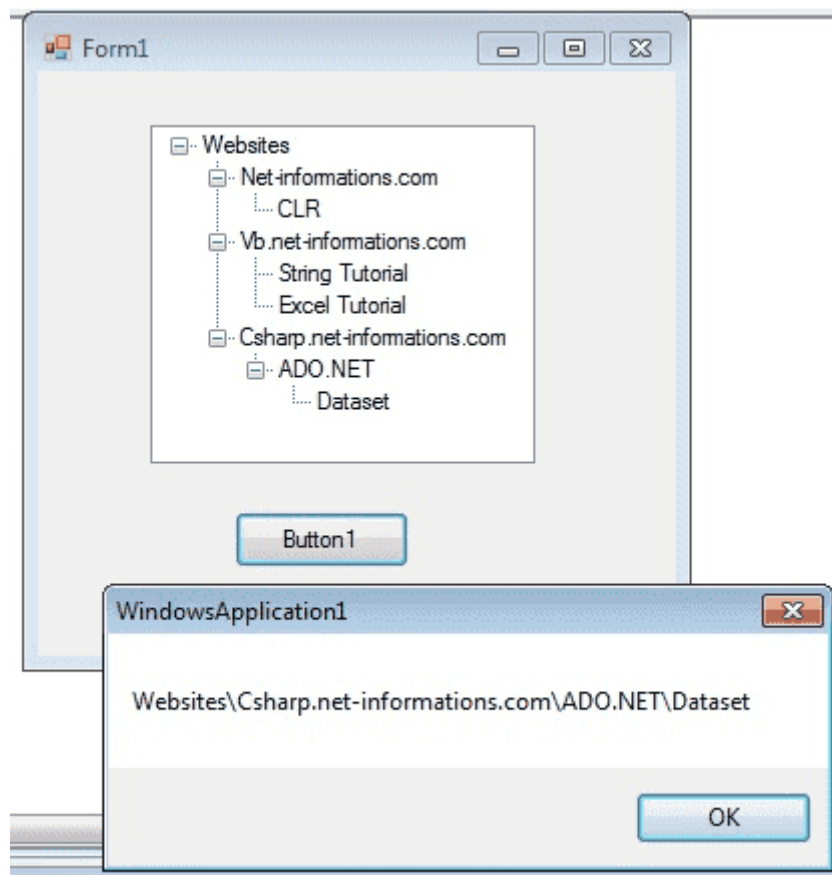
12.53. TreeView

<https://youtu.be/PNIZDLFPmXE>

<http://www.yevol.com/en/vcsharp/applicationdesign/lesson31.htm>

C# Treeview Control

The TreeView control contains a hierarchy of TreeViewItem controls. It provides a way to display information in a hierarchical structure by using collapsible nodes. The top level in a tree view are root nodes that can be expanded or collapsed if the nodes have child nodes.



You can explicitly define the TreeView content or a data source can provide the content. The user can expand the TreeNode by clicking the plus sign (+) button, if one is displayed next to the TreeNode, or you can expand the TreeNode by calling the TreeNode.Expand method. You can also navigate through tree views with various

properties: FirstNode, LastNode, NextNode, PrevNode, NextVisibleNode, PrevVisibleNode.

The fullpath method of treeview control provides the path from root node to the selected node.

```
treeView1.SelectedNode.FullPath.ToString ();
```

Tree nodes can optionally display check boxes. To display the check boxes, set the CheckBoxes property of the TreeView to true.

```
treeView1.CheckBoxes = true;
```

The following C# program shows a simple demonstration of treeview control

```
using System;
using System.Drawing;
using System.Windows.Forms;
namespace WindowsFormsApplication1
{
    public partial class Form1 : Form
    {
        public Form1()
        {
            InitializeComponent();
        }
        private void Form1_Load(object sender, EventArgs e)
        {
            TreeNode tNode ;
            tNode = treeView1.Nodes.Add("Websites");
            treeView1.Nodes[0].Nodes.Add("Net-informations.com");
            treeView1.Nodes[0].Nodes[0].Nodes.Add("CLR");
            treeView1.Nodes[0].Nodes.Add("Vb.net-informations.com");
            treeView1.Nodes[0].Nodes[1].Nodes.Add("String Tutorial");
            treeView1.Nodes[0].Nodes[1].Nodes.Add("Excel Tutorial");
            treeView1.Nodes[0].Nodes.Add("Csharp.net-informations.com");
            treeView1.Nodes[0].Nodes[2].Nodes.Add("ADO.NET");
            treeView1.Nodes[0].Nodes[2].Nodes[0].Nodes.Add("Dataset");
        }
        private void button1_Click(object sender, EventArgs e)
        {
            MessageBox.Show(treeView1.SelectedNode.FullPath.ToString ());
        }
    }
}
```

[Download Source Code](#)

[Print Source Code](#)

Source: [\[#\] C# Treeview Control](#) (24 August 2015 3:35 AM)

=====

Published on Jun 10, 2013

C# TreeView Tutorial

Populate a tree view in c#

Focus selected node in treeview by c#

Extending the treenode in the treeview (C#)

How to ADD child Nodes to Treeview in WPF, using C# code.

C# 2008 - Basic use of a TreeView control

c# - How to use TreeView

How to: Add and Remove Nodes with the Windows Forms TreeView

c# - How to Efficiently Delete Checked Items from a TreeView

c# - Removing nodes from a TreeView with multiple selection

c# - Delete selected parent node and child nodes

c# - How do I clear the selection of a selected node in a TreeView

Searches related to how to use treeview in c#

how to use treeview in c# windows application

c# treeview example

windows forms treeview

winforms treeview

c# custom treeview

c# treeview clear all nodes

csharp treeview example

visual studio treeview

```
using System;
using System.Drawing;
using System.Windows.Forms;
namespace WindowsFormsApplication1
{
public Form1()
    {
        InitializeComponent();

        private void button1_Click(object sender, EventArgs e)
        {
            d
                TreeNode treeView = new TreeNode();
                treeView.Nodes.Add("Books");
                treeView.Nodes.Add("Papers");
                treeView.Nodes.Add("Folders");

                treeView.Nodes[0].Nodes.Add("Book1");
                treeView.Nodes[0].Nodes.Add("Book2");
            }

            /*
            treeview1.SelectedMode.Remove();
            treeview1.Nodes.Clear();
            checkboxes
            */
        }
    }
}
```

Source: [\[#\] C# Treeview Control](#) (20 July 2015 1:28 PM)

Source: [\[#\] C# Treeview Control](#) (20 July 2015 1:23 PM)

Source: [\[#\] C# Tutorial 72: How to use TreeView \(Add ,Removes Clear , Delete Checked Items from a TreeView\) - YouTube](#) (20 July 2015 12:51 PM)

Source: [\[#\] C# Tutorial 72: How to use TreeView \(Add ,Removes Clear , Delete Checked Items from a TreeView\) - YouTube](#) (20 July 2015 12:50 PM)

12.54. VScrollBar

12.55. WebBrowser

<https://www.youtube.com/watch?v=ezu0zDTixn8>
<http://www.dotnetperls.com/webbrowser>

```
webBrowser1.Navigate(textBox1.Text);
```

```
GetNextControl();  
GetPreferredSize  
GetType  
GoForward  
GoBack  
GoHome  
GoSearch  
GotFocus  
Handle
```

12.56. Menus and Toolbars

To Add a Top Menu

MenuStrip from toolbox

Type here to add items

'-' results in break line

'&' before character makes that character a shortcut

Properties: RenderMode - System

Quit menu option

Application.Exit(); Will cause program to exit private void quitToolStripMenuItem_Click(object sender, EventArgs e)
e.g.

```
{  
    if (MessageBox .Show("Really Quit?", "Exit", MessageBoxButtons.OKCancel) == DialogResult .OK)  
    {  
        Application.Exit();  
    }  
}
```

Source: [To Add a Top Menu](#) (17 December 2013 11:44 AM)

13. Common

13.1. Random

Random r = new Random();

To generate random number between 2 values;

numb = r.Next(1, 101);

range arguments (lowest, high+1:

If highs and lows are not set, the number generated will be between 0 and 2147483647.

Seeds

A value for generating random numbers. If not set, the default value is determined by the current system time. If the seed is set, the same random numbers will be generated.

e.g. Random r = new Random(seed, e.g.5);

Generate a random number between 0 and 1.0. (not including 1.0)

```
Random r = new Random();  
double numb = r.NextDouble();  
MessageBox.Show($"Double Number = {numb}");
```

13.1.1. Sample

To get random numbers, use Random

Random r = new Random();
parameter can be empty for computer generated seed
specified number to generate repeated sequence

```
int num = r.Next(n1,n2);  
n1 is the minimum wanted  
n2 is maximum wanted plus 1
```

=====

Example:

In coin flip, what is outcome after 100 flips

```
private void button1_Click(object sender, EventArgs e)  
{  
    // listBox1.Items.Clear();  
    int heads = 0;  
    int tails = 0;  
    int toss = 0;  
  
    Random r = new Random();  
    for(int i = 0; i < 100; i++)  
    {  
        toss = r.Next(1, 3);  
        if(toss == 1) heads++;  
        if(toss == 2) tails++;  
    }  
    listBox1.Items.Add($"Heads = {heads.ToString()}");
```

```

        listBox1.Items.Add($"Tails = {tails.ToString()}");
    }

```

13.2. Calling Another Program

```
using System.Diagnostics;
```

```
Process notePad = new Process();
```

```

notePad.StartInfo.FileName = "notepad.exe";
notePad.StartInfo.Arguments = "ProcessStart.cs";
process.WaitForExit();
notePad.Start();

```

Source: [using System.Diagnostics;](#) (17 December 2013 11:20 AM)

13.3. Log File

```

public void errors(string logItem)
{
    if(!File.Exists(LogPath))
    {
        string s = "Missing cities\n\n";
        File.WriteAllText(LogPath, s);
    }
    logItem = logItem + "\n";
    File.AppendAllText(LogPath, logItem);
}

```

Source: [public void errors\(string logItem\)](#) (17 December 2013 11:21 AM)

13.4. CHM Files

```
Help.ShowHelp(this, "file://c:\\test.chm");
```

```

string h = "file://" + s;
Help.ShowHelp(this, h);

```

13.5. Startup

```

public Form1()
{
    InitializeComponent();

    string path = Directory.GetCurrentDirectory();
    bool test = path.Contains("Filename\\bin");

    if (test == true) textBox1.Text = I.Folder;
    if (test == false)
    {
        textBox1.Text = path;
        I.chgPara("Folder=", textBox1.Text);
        I.Folder = textBox1.Text;
    }
}

```

```
    }
}
```

13.6. INI File

```
private void button2_Click(object sender, EventArgs e)
{
    string user = Environment.UserName;
    string Path = "c:\\Users\\" + user + "\\AppData\\local\\" + "Bintz" + "\\"
    bool x = Directory.Exists(Path);
    if (x == false) Directory.CreateDirectory(Path);

    Process notePad = new Process();
    notePad.StartInfo.FileName = "notepad.exe";
    notePad.StartInfo.Arguments = Path + "toc.ini";
    notePad.Start();
}
```

13.6.1. Get Value

```
public string getParameter(string targ, string INIFile)
{
    StreamReader rp;

    if (File.Exists(INIFile) == true)
    {
        int x = -1;
        rp = new StreamReader(INIFile);
        string buf = "";
        while (rp.Peek() != -1)
        {
            string s = rp.ReadLine();
            if (s.Contains(targ) == true)
            {
                x = s.IndexOf("=");
                buf = s.Substring(x + 1);
            }
        }

        rp.Close();
        rp.Dispose();
        return buf;
    }
    else return ("-1");
}
```

13.6.2. Change Values

```
public void changeParameter(string targ, string key, int home)
{
    StreamReader rp;
    StreamWriter wp;
    string rfn = path + AppName + ".ini";
    string wfn = path + "xxxtemp";
    string backup = path + AppName + ".bak";
    string CWD = Directory.GetCurrentDirectory();

    if (File.Exists(rfn) == true)
    {
```

```

rp = new StreamReader(rfn);
wp = new StreamWriter(wfn);
while (rp.Peek() != -1)
{
    string s = rp.ReadLine();
    if (s.Contains(targ) == true)
    {
        if (home == 0) s = targ + "=" + key;
        if (home == 1) s = targ + "=" + CWD;
    }
    wp.WriteLine(s);
}

rp.Close();
wp.Close();
rp.Dispose();
wp.Dispose();
File.Replace(wfn, rfn, backup);
}

```

13.7. Misc

Remove charater from string

```
s = s.Replace("\t", "");
```

Access all files in a directory

```
string[] fileEntries = Directory.GetFiles(directoryName);
foreach (string fileName in fileEntries)
```

Convert string to int

```
int x = Convert.ToInt32("123");
```

string path =

```
Environment.GetFolderPath(Environment.SpecialFolder.ApplicationData); ;
```

14. Graphics

14.1. Paint Forms

Screen Location specified as Point(x,y)

Point(0,0) top left corner

ClientRectangle.Width

ClientRectangle.Height

to get into graphics

1. Click once on form

2. In Properties Header click on lightning bolt

3. In Property Box choose paint

Should result in:

```
private void Form1_Paint(object sender, PaintEventArgs e)
```

```
Graphics g = e.Graphics;
```

```
// Create a font
```

```
Font f = new Font("Verdana",30);
```

```
// Create a brush
```

```
SolidBrush b = new SolidBrush(Color.Blue);
```

```
// Draw a string
```

```
g.DrawString("Hello World", f, b, 40, 40); // 40 40 is location point
```

```
// Create a pen
```

```
Pen p = new Pen(Color.Red, 4); // 4 is thickness
```

```
// Draw a line
```

```
g.DrawLine(p, 0, 0, 140, 140); // starting/ending points
```

```
// Draw a rectangle
```

```
g.DrawRectangle(p, 50, 50, 20, 20); // top-left, bottom-right
```

```
// Draw filled in rectangle
```

```
SolidBrush s = new SolidBrush(Color.Purple);
```

```
g.FillRectangle(s, 120, 40, 50, 50); // location, size
```

```
// Draw a Circle
```

```
SolidBrush snew = new SolidBrush(Color.DarkBlue);
```

```
g.FillEllipse(snew, 100, 40, 20, 20);
```

```
string StartUpPath = Application.StartupPath;
```

```
string f = StartUpPath + @"\sjbspring.jpg";
```

```
Image image = Image.FromFile( f );
```

```
g.DrawImage(image, 370, 5, 50, 50);
```

```
//Dispose all objects
```

```
font.Dispose();
```

```
brush.Dispose();
```

```
pen.Dispose();
```

```
mypen.Dispose();
```

```
newpen.Dispose();
```

```
solidbrush.Dispose();
```

14.2. Image Manipulation

Creating a color filter is a fantastic way to understand how digital images are structured. Every color you see on a screen is just a combination of Red, Green, and Blue (RGB). By mathematically tweaking these values, you can create various visual effects.

To manipulate an image, you'll iterate through every pixel, pull its color, modify the values, and save it back.

Popular Filter Logic

Here are three classic filters you can implement by modifying the RGB channels of each pixel:

Filter	Logic / Formula	Effect
---	---	---
Invert	Subtract current RGB from 255.	Turns colors into their opposites (negatives).
Sepia	Weighted formula adding more Red/Yellow.	Gives photos an antique, brownish look.
Brightness	Add a constant value to R, G, and B.	Makes the image lighter or darker.

C# Implementation: The Inversion Filter

Inverting an image is the easiest to code because you simply subtract each color component from the maximum value ().

```
```\ncsharp
using System.Drawing;

public void InvertImage(string inputPath, string outputPath)
{
 // 1. Load the original image
 Bitmap bmp = new Bitmap(inputPath);

 // 2. Loop through every row and column of pixels
 for (int y = 0; y < bmp.Height; y++)
 {
 for (int x = 0; x < bmp.Width; x++)
 {
 // Get the color of the current pixel
 Color oldColor = bmp.GetPixel(x, y);

 // Subtract from 255 to get the opposite color
 int newR = 255 - oldColor.R;
 int newG = 255 - oldColor.G;
 int newB = 255 - oldColor.B;

 // 3. Create the new color and apply it
 Color newColor = Color.FromArgb(oldColor.A, newR, newG, newB);
 bmp.SetPixel(x, y, newColor);
 }
 }

 // 4. Save the result
 bmp.Save(outputPath);
}

```\n
```

A Challenge: The Sepia Filter

The **Sepia** filter is slightly more complex because it uses a specific "weight" for each color to mimic old photographic film. The standard formula is:

> **Pro Tip:** When doing these calculations, always ensure your final value doesn't exceed 255 (the maximum for a byte), or your code will crash! You can use `Math.Min(255, result)` to keep it safe.

Would you like to try writing the logic for the **Sepia** filter using that formula, or should we look at how to make this code run faster using **LockBits**?